Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

WINTER SEMESTER 2025-26

ASSESSMENT - 2

Name: Jaswanth Pothuru

Registration Number: 23MIS0178

School: School of Computer Science Engineering and Information Systems (SCORE)

Course Code: ISWE406P

Course Title: Agile Development Process And Devops Lab

Faculty: Dr. Ramalingam M

Slot: L57+L58

Experiment 1

Develop a simple application (console-based or GUI-based) using any programming language of your choice. Initialize a local Git repository for the project and manage the source code using Git version control. Commit the application source code with proper commit messages and maintain version history.

Create a remote repository on GitHub and push the local repository to GitHub. Verify successful hosting by accessing the repository online.

Perform branch-related operations by creating at least one feature branch from the main branch, implement a minor enhancement or modification in the feature branch, commit the changes, and merge the feature branch back into the main branch. Resolve any merge conflicts if they arise and push the updated main branch to GitHub.

Step 1: Project Creation

A project folder named **SCM_Experiment1** was created and opened using Visual Studio Code. A Java source file named **Calculator.java** was created and the calculator program was written.

Step 2: Git Repository Initialization

The local Git repository was initialized using the following command:

“git init”

Step 3: Checking Repository Status

The status of the repository was verified:

“git status”

Step 4: Adding Files to Staging Area

The source file was added to the staging area:

“git add Calculator.java”

Step 5: Committing the Source Code

The staged file was committed with an appropriate commit message: “git commit -m

"Initial commit: Calculator application"”

Step 6: Branch Configuration

The default branch name was changed to main:

“git branch -M main”

Step 7: Creating Remote Repository

A new repository was created on GitHub and linked to the local repository:

“git remote add origin jaswanthhitman45/calculator”

Step 8: Pushing Code to GitHub

The local repository was pushed to GitHub:

“git push -u origin main”

```
PS C:\Users\Lenovo\Desktop\SimpleCalculator> git init
>>
Initialized empty Git repository in C:/Users/Lenovo/Desktop/SimpleCalculator/.git/
PS C:\Users\Lenovo\Desktop\SimpleCalculator> git status
>>
No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        calculator.cpp

nothing added to commit but untracked files pgit add calculator.cppto track)
>> C:\Users\Lenovo\Desktop\SimpleCalculator>
PS C:\Users\Lenovo\Desktop\SimpleCalculator> git commit -m "Initial commit: Added basic calculator for addition"
>>
[master (root-commit) 69ec4fc] Initial commit: Added basic calculator for addition
 1 file changed, 11 insertions(+)
 create mode 100644 calculator.cpp
PS C:\Users\Lenovo\Desktop\SimpleCalculator> git log
>>
commit 69ec4fc841cd5c636c85b8495b454f289d5c0c1c (HEAD -> master)
Author: POTHURU-JASWANTH <pothuru.jaswanth2023@vitstudent.ac.in>
Date:   Thu Jan 8 18:06:07 2026 +0530

    Initial commit: Added basic calculator for addition
```

```
PS C:\Users\Lenovo\Desktop\SimpleCalculator> git branch -M main
PS C:\Users\Lenovo\Desktop\SimpleCalculator> git remote add origin https://github.com/jaswanthhitman45/calculator.git
PS C:\Users\Lenovo\Desktop\SimpleCalculator> git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 400 bytes | 100.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/jaswanthhitman45/calculator.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Step 9: Feature Branch Creation

A new branch was created to add an enhancement:

“git checkout -b feature-division”

Step 10: Enhancement Implementation

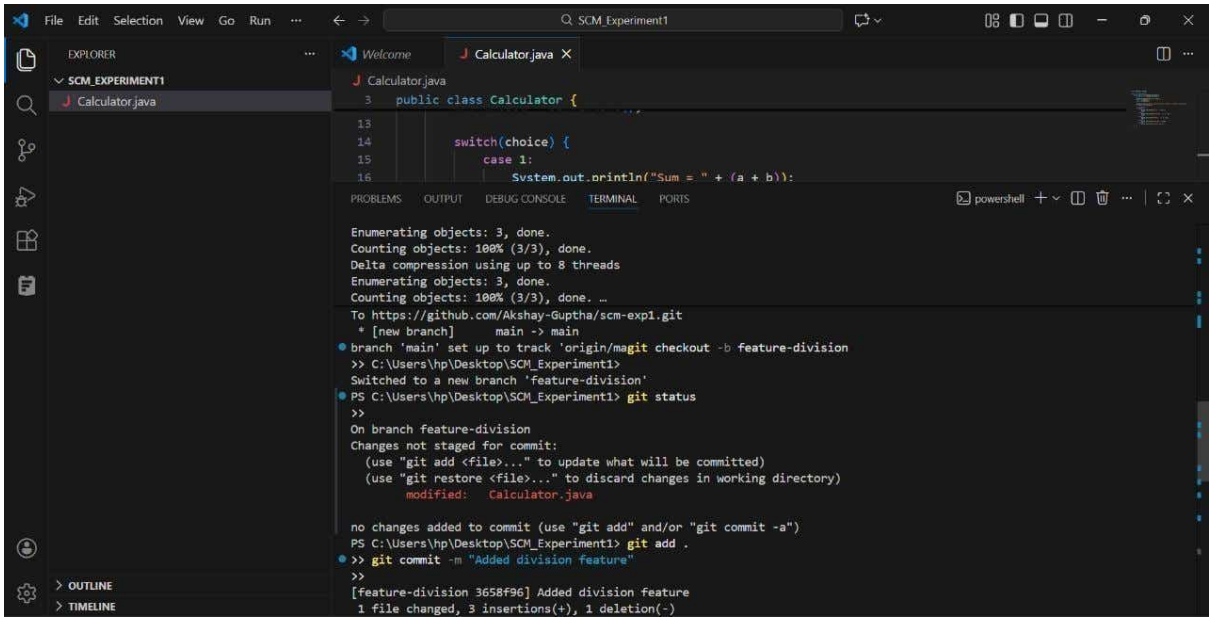
A division operation was added to the calculator program.

Step 11: Commit Feature Changes

The modified file was staged and committed:

“git add .”

“git commit -m "Added division feature"”



Step 12: Merging Feature Branch

The branch was switched back to main and merged:

“git checkout main git

merge feature-division”

Step 13: Final Push

The updated main branch was pushed to GitHub:

“git push origin main”

```
PS C:\Users\Lenovo\Desktop\SimpleCalculator> git checkout -b feature-subtraction
>>
Switched to a new branch 'feature-subtraction'
PS C:\Users\Lenovo\Desktop\SimpleCalculator> git branch
>>
* feature-subtraction
  main
PS C:\Users\Lenovo\Desktop\SimpleCalculator> git add calculator.cpp
PS C:\Users\Lenovo\Desktop\SimpleCalculator> git commit -m "Added subtraction functionality"
[feature-subtraction f8642d0] Added subtraction functionality
 1 file changed, 1 insertion(+)
PS C:\Users\Lenovo\Desktop\SimpleCalculator> git push origin feature-subtraction
>>
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 369 bytes | 123.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'feature-subtraction' on GitHub by visiting:
remote:      https://github.com/jaswanthhitman45/calculator/pull/new/feature-subtraction
remote:
To https://github.com/jaswanthhitman45/calculator.git
 * [new branch]      feature-subtraction -> feature-subtraction
```

```
PS C:\Users\Lenovo\Desktop\SimpleCalculator> git checkout main
>>
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
PS C:\Users\Lenovo\Desktop\SimpleCalculator> git merge feature-subtraction
>>
Updating 69ec4fc..f8642d0
Fast-forward
 calculator.cpp | 1 +
 1 file changed, 1 insertion(+)
PS C:\Users\Lenovo\Desktop\SimpleCalculator> git push origin main
>>
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/jaswanthhitman45/calculator.git
   69ec4fc..f8642d0  main -> main
PS C:\Users\Lenovo\Desktop\SimpleCalculator>
```

Experiment 2

Form a team of five members to collaboratively develop a simple application using Gitand GitHub. One member shall act as the repository owner, while the remaining members shall act as collaborators.

Create a shared GitHub repository and clone it to all team members' local systems. Each team member must create a separate feature branch from the main branch and modify the same source file by adding their own functionality or contribution.

Each collaborator shall commit and push their changes to their respective branches and raise a pull request to the main branch. The repository owner shall review the pull requests, resolve any merge conflicts that arise due to simultaneous modification of the same file, and merge the changes into the main branch. Finally, push the updated main branch to GitHub and verify that the final application includes contributions from all five team members. Document the Git commands used and explain the collaborative workflow followed in the experiment. **Team Details**

- **Repository Owner:** Marimamula Akshay Guptha
- **Collaborators:**
    1. Jaswanth Pothuru
    2. Dhanush Reddy
    3. Koyahari Krishna
    4. Praneeth Tullala

Step 1: Repository Creation The repository owner created a GitHub repository named **scm-team-exp** and added collaborators using the *Manage Access* option.

Step 2: Cloning the Repository

The repository was cloned into the local system using Visual Studio Code:

"git clone https://github.com/Akshay-Guptha/scm-team-exp.git"

Step 3: Base Application Creation

The owner created the base Java file **TeamApp.java** and added the initial code.

Step 4: Committing and Pushing Base Code

The base application was staged, committed, and pushed to the main branch:

"git add TeamApp.java"

"git commit -m "Base team application""

"git push -u origin main"

Step 5: Feature Branch Creation by Collaborator

Each collaborator created a separate feature branch from the main branch:

"git checkout -b feature-member1"

Step 6: Contribution by Team Member

The collaborator modified the same file **TeamApp.java** by adding their own contribution message.

Step 7: Commit and Push by Collaborator

The changes were committed and pushed to the respective feature branch:

“git add .”

“git commit -m "Added contribution by Member 1"”

“git push origin feature-member1”

Step 8: Pull Request Creation

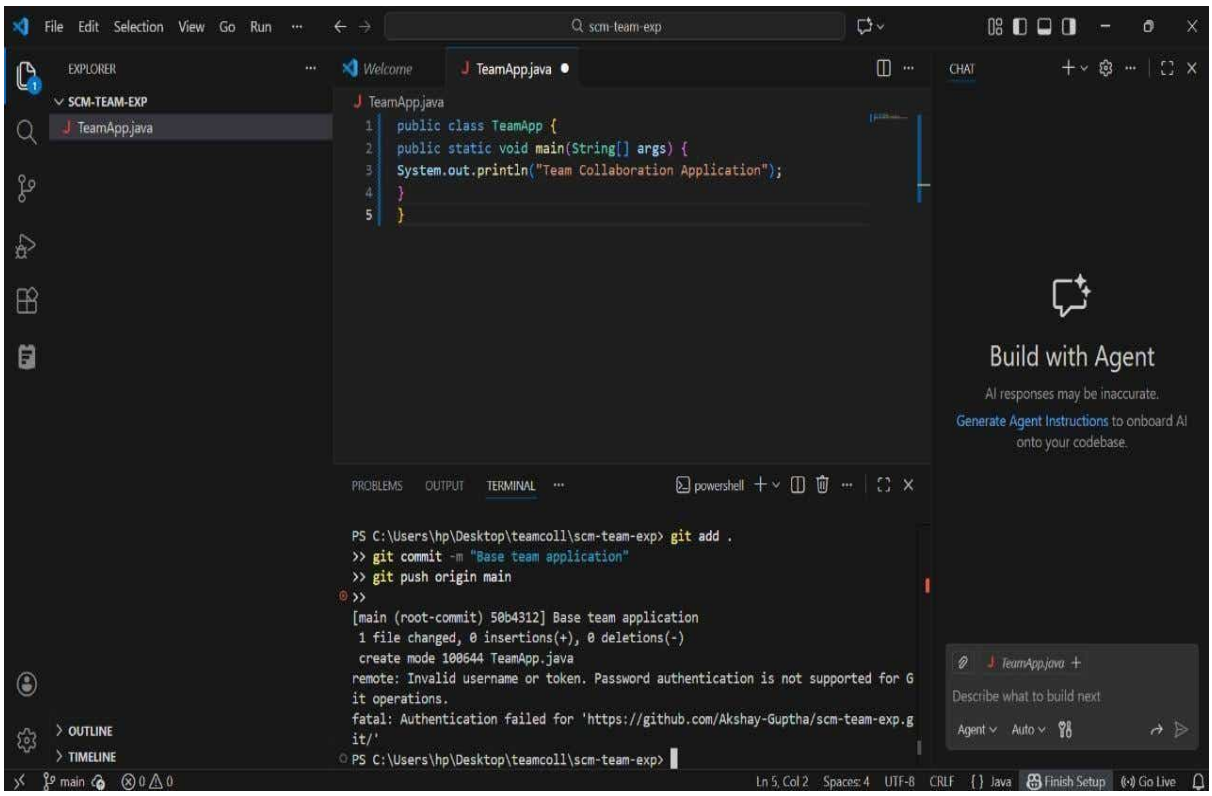
The collaborator raised a pull request from feature-member1 to the main branch using GitHub.

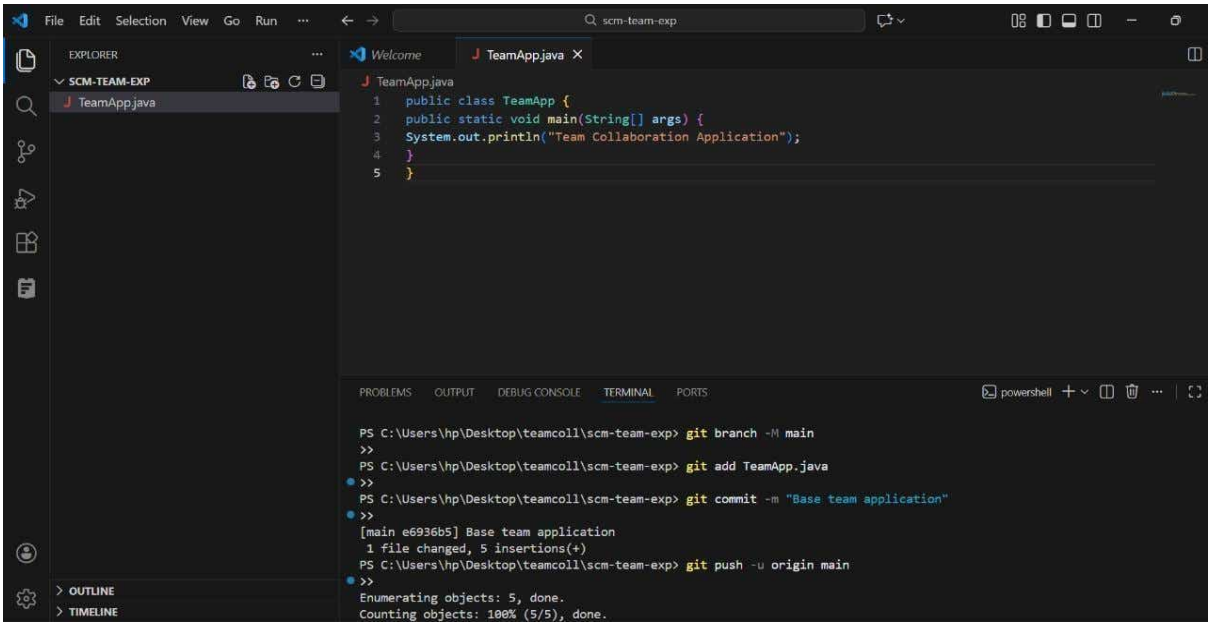
Step 9: Review and Merge by Owner

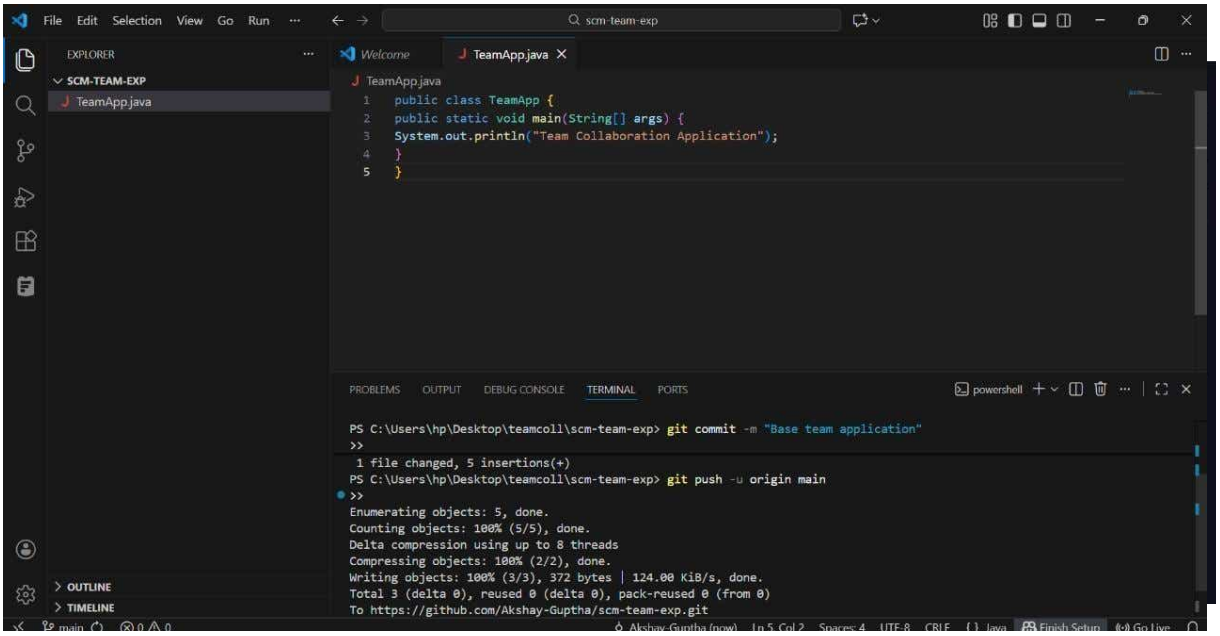
The repository owner reviewed the pull request and successfully merged it into the main branch.

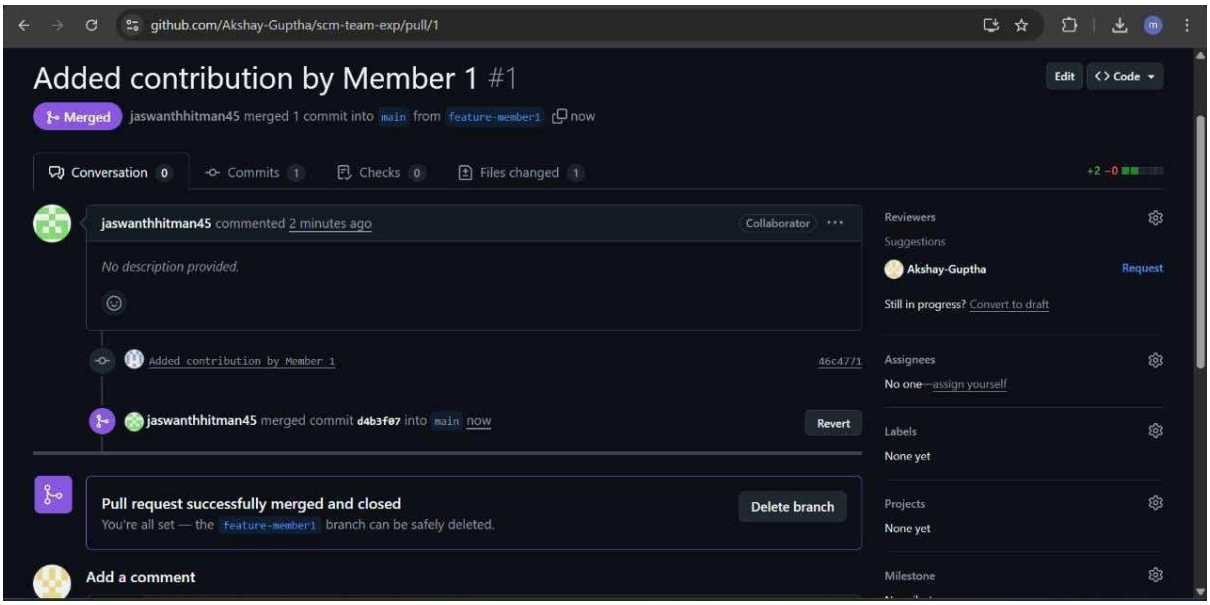
Step 10: Final Verification

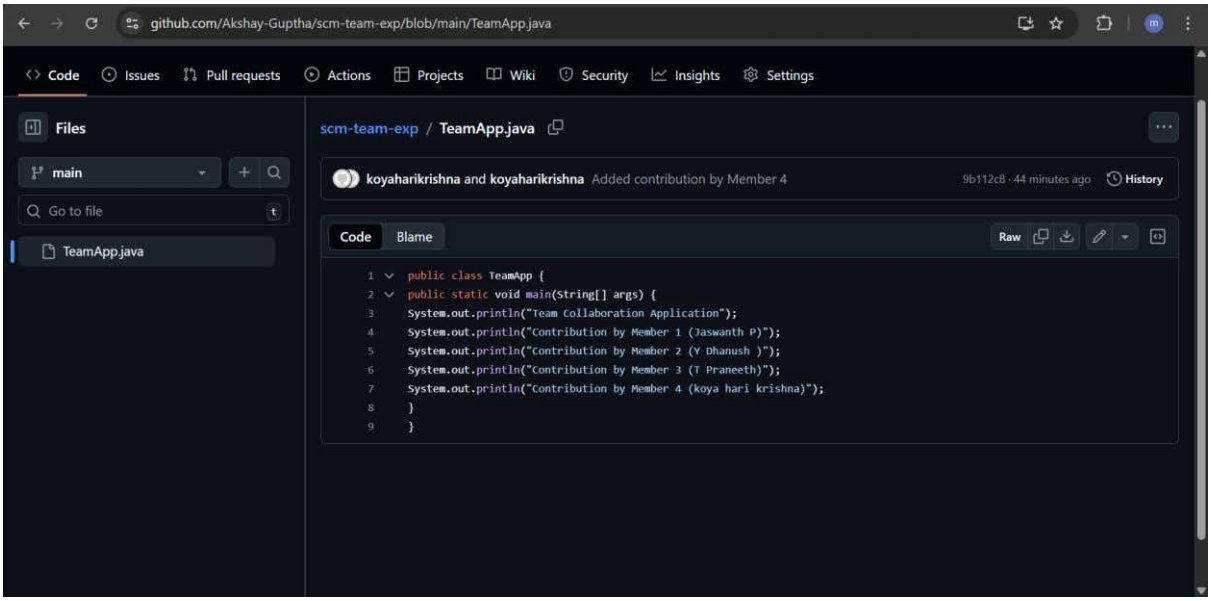
After merging, the main branch contained the base code along with the collaborator’s contribution.

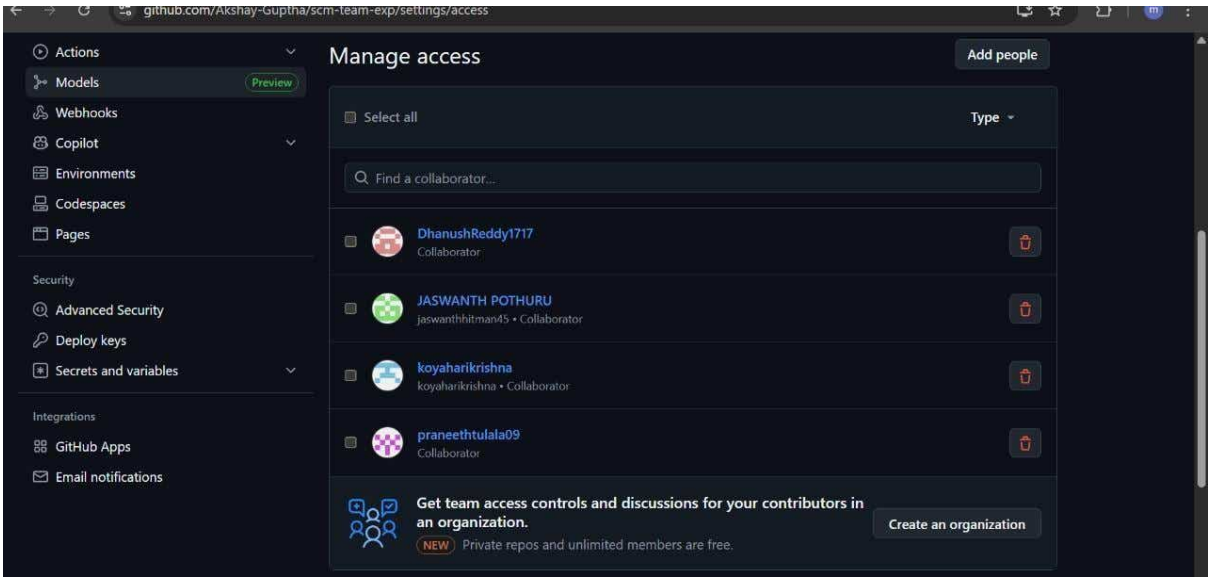



File Edit Selection View Go Run
scm-team-exp
EXPLORER
SCM-TEAM-EXP
TeamApp.java
Welcome
TeamApp.java
public class TeamApp {
public static void main(String[] args) {
System.out.println("Team Collaboration Application");
}
}
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
powershell
PS C:\Users\hp\Desktop\teamcoll\scm-team-exp> git branch -M main
>>
PS C:\Users\hp\Desktop\teamcoll\scm-team-exp> git add TeamApp.java
>>
PS C:\Users\hp\Desktop\teamcoll\scm-team-exp> git commit -m "Base team application"
>>
[main e6936b5] Base team application
1 file changed, 5 insertions(+)
PS C:\Users\hp\Desktop\teamcoll\scm-team-exp> git push -u origin main
>>
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
OUTLINE
TIMELINE


File Edit Selection View Go Run
scm-team-exp
EXPLORER
SCM-TEAM-EXP
TeamApp.java
Welcome
TeamApp.java
public class TeamApp {
public static void main(String[] args) {
System.out.println("Team Collaboration Application");
}
}
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
powershell
PS C:\Users\hp\Desktop\teamcoll\scm-team-exp> git commit -m "Base team application"
>>
1 file changed, 5 insertions(+)
PS C:\Users\hp\Desktop\teamcoll\scm-team-exp> git push -u origin main
>>
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 372 bytes | 124.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Akshay-Guptha/scm-team-exp.git
OUTLINE
TIMELINE


github.com/Akshay-Guptha/scm-team-exp/pull/1
Added contribution by Member 1 #1
Edit
Code
Merged
jaswanthhitman45 merged 1 commit into main from feature-member1 now
Conversation 0
Commits 1
Checks 0
Files changed 1
+2 -0
jaswanthhitman45 commented 2 minutes ago
Collaborator
No description provided.
Added contribution by Member 1
46c4771
jaswanthhitman45 merged commit d4b3f07 into main now
Revert
Pull request successfully merged and closed
You're all set — the feature-member1 branch can be safely deleted.
Delete branch
Add a comment
Reviewers
Suggestions
Akshay-Guptha
Request
Still in progress? Convert to draft
Assignees
No one—assign yourself
Labels
None yet
Projects
None yet
Milestone


github.com/Akshay-Guptha/scm-team-exp/blob/main/TeamApp.java
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
Files
main
Go to file
TeamApp.java
scm-team-exp / TeamApp.java
koyaharikrishna and koyaharikrishna Added contribution by Member 4
9b112c8 · 44 minutes ago
History
Code
Blame
Raw
public class TeamApp {
public static void main(String[] args) {
System.out.println("Team Collaboration Application");
System.out.println("Contribution by Member 1 (Jaswanth P)");
System.out.println("Contribution by Member 2 (Y Dhanush)");
System.out.println("Contribution by Member 3 (T Praneeth)");
System.out.println("Contribution by Member 4 (koya hari krishna)");
}
}


github.com/Akshay-Guptha/scm-team-exp/settings/access
Actions
Models
Preview
Webhooks
Copilot
Environments
Codespaces
Pages
Security
Advanced Security
Deploy keys
Secrets and variables
Integrations
GitHub Apps
Email notifications
Manage access
Add people
Select all
Type
Find a collaborator...
DhanushReddy1717
Collaborator
JASWANTH POTHURU
jaswanthhitman45 • Collaborator
koyaharikrishna
koyaharikrishna • Collaborator
praneethtulala09
Collaborator
Get team access controls and discussions for your contributors in an organization.
NEW
Private repos and unlimited members are free.
Create an organization

GIT COMMANDS AND CASE STUDIES

Case Study 1: New Project Version Control Setup

Scenario: You are assigned to develop a Library Management System. Before coding, your team decides to enable version control.

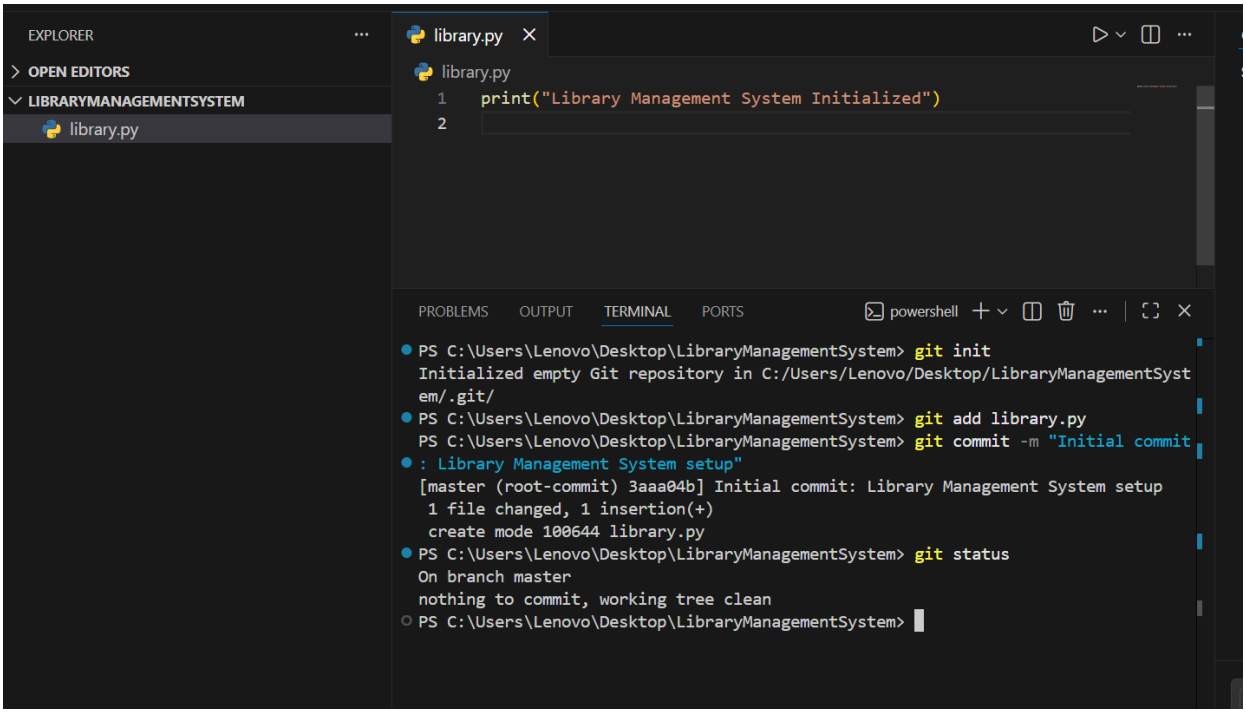
Tasks:

- Create a project folder and open it in VS Code

- Initialize a Git repository

- Create a Python file and track it using Git

- Commit the initial version



Case Study 2: Tracking Daily Code Changes

Scenario: You are working on a Student Registration Module. You make small updates daily and need to record them.

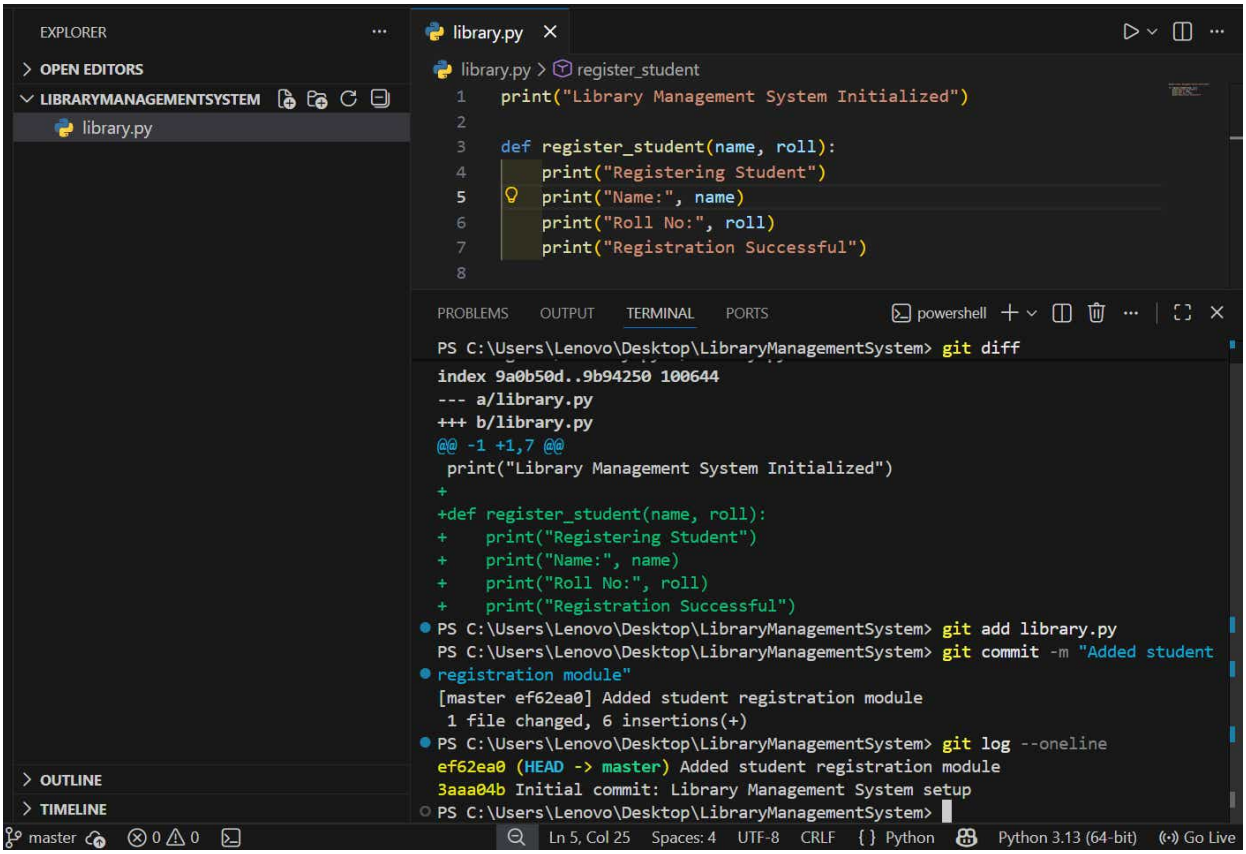
Tasks:

- Modify an existing file

- Check file differences

- Stage and commit changes

- View commit history



Case Study 3: Feature Development Using Branches

Scenario: A new Login feature must be developed without disturbing main code.

Tasks:

- Create a new branch

- Add login-related code

- Commit changes

- Switch between branches

```
def login(username, password):
    print("Checking credentials")
    if username == "admin" and password == "admin123":
        print("Login Successful")
    else:
        print("Invalid Login")
```

```
PS C:\Users\Lenovo\Desktop\LibraryManagementSystem> git checkout login-feature
>>
Switched to branch 'login-feature'
PS C:\Users\Lenovo\Desktop\LibraryManagementSystem> git status
>>
On branch login-feature
nothing to commit, working tree clean
PS C:\Users\Lenovo\Desktop\LibraryManagementSystem> git add library.py
PS C:\Users\Lenovo\Desktop\LibraryManagementSystem> git commit -m "Added login fe
ature"
On branch login-feature
nothing to commit, working tree clean
PS C:\Users\Lenovo\Desktop\LibraryManagementSystem>
```

Case Study 4: Merging Feature into Main Code Scenario:

The login feature is tested and approved.

Tasks:

- Switch to main branch

- Merge feature branch

- Verify merged content

```
def login(username, password):
    print("Checking credentials")
    if username == "admin" and password == "admin123":
        print("Login Successful")
    else:
        print("Invalid Login")
```

```
PS C:\Users\Lenovo\Desktop\LibraryManagementSystem> git checkout master
>>
Switched to branch 'master'
PS C:\Users\Lenovo\Desktop\LibraryManagementSystem> git merge login-feature
>>
Updating ef62ea0..756e234
Fast-forward
 library.py | 7 +++++++
 1 file changed, 7 insertions(+)
PS C:\Users\Lenovo\Desktop\LibraryManagementSystem> git log --oneline
>>
756e234 (HEAD -> master, login-feature) Added login feature
ef62ea0 Added student registration module
3aaa04b Initial commit: Library Management System setup
PS C:\Users\Lenovo\Desktop\LibraryManagementSystem>
```

Case Study 5: Working with Remote Repository

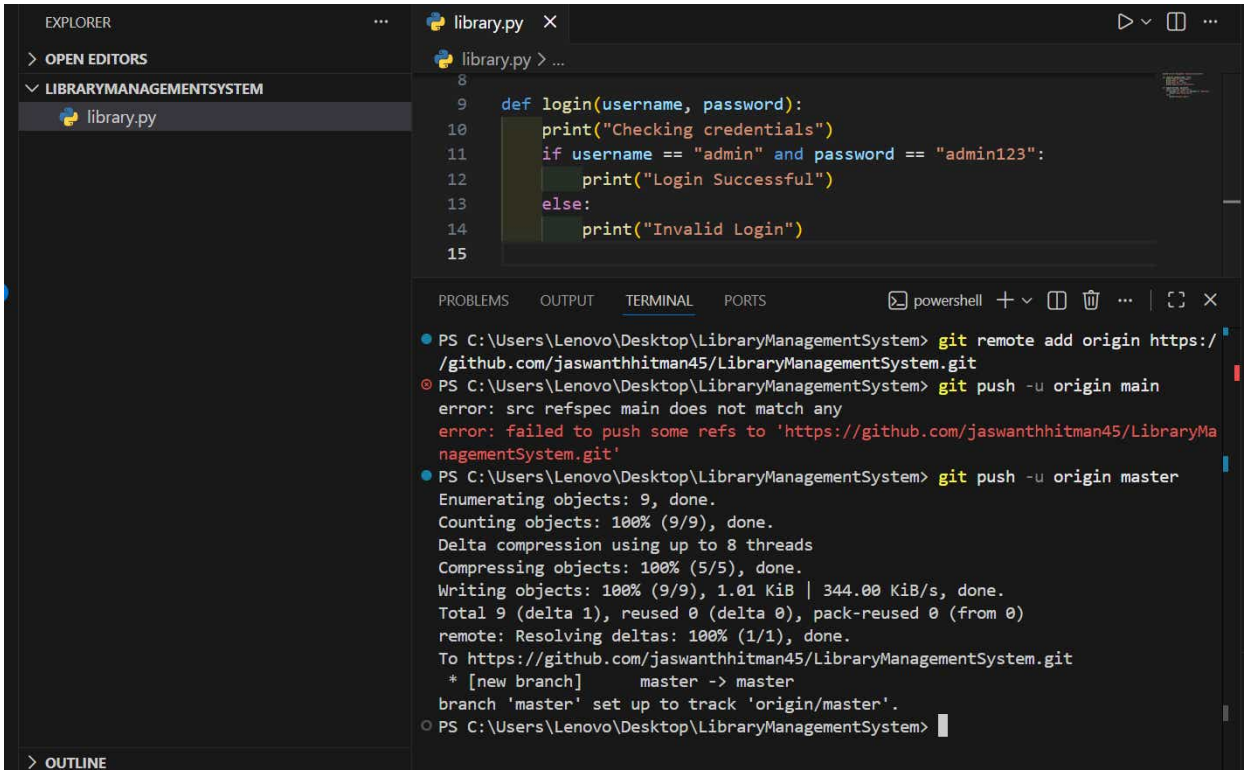
Scenario: The project must be shared using GitHub.

Tasks:

- Create remote repository

- Add remote origin

- Push code to GitHub



Case Study 6: Undoing Mistakes Scenario:

A file is modified incorrectly.

Tasks:

- View file changes

- Discard uncommitted changes

EXPLORER

OPEN EDITORS

LIBRARYMANAGEMENTSYSTEM

library.py M

library.py M

```
print("WRONG CODE ADDED BY MISTAKE")
```

PROBLEMS OUTPUT TERMINAL PORTS

```
PS C:\Users\Lenovo\Desktop\LibraryManagementSystem> git diff
diff --git a/library.py b/library.py
index 095aaed..c7f5154 100644
--- a/library.py
+++ b/library.py
@@ -1,14 +1 @@
-print("Library Management System Initialized")
-
-def register_student(name, roll):
-    print("Registering Student")
-    print("Name:", name)
-    print("Roll No:", roll)
-    print("Registration Successful")
-
-def login(username, password):
-    print("Checking credentials")
-    if username == "admin" and password == "admin123":
-        print("Login Successful")
-    else:
-        print("Invalid Login")
+print("WRONG CODE ADDED BY MISTAKE")
```

OUTLINE

TIMELINE

EXPLORER

OPEN EDITORS

LIBRARYMANAGEMENTSYSTEM

library.py

library.py > ...

```
def login(username, password):
    print("Checking credentials")
    if username == "admin" and password == "admin123":
        print("Login Successful")
    else:
        print("Invalid Login")
```

PROBLEMS OUTPUT TERMINAL PORTS

```
PS C:\Users\Lenovo\Desktop\LibraryManagementSystem> git diff
@@ -1,14 +1 @@
-print("Library Management System Initialized")
-
-def register_student(name, roll):
-    print("Registering Student")
-    print("Name:", name)
-    print("Roll No:", roll)
-    print("Registration Successful")
-
-def login(username, password):
-    print("Checking credentials")
-    if username == "admin" and password == "admin123":
-        print("Login Successful")
-    else:
-        print("Invalid Login")
+print("WRONG CODE ADDED BY MISTAKE")
PS C:\Users\Lenovo\Desktop\LibraryManagementSystem> git checkout -- library.py
PS C:\Users\Lenovo\Desktop\LibraryManagementSystem> git status
On branch master
Your branch is up to date with 'origin/master'.
```

OUTLINE

TIMELINE

Case Study 7: Handling Merge Conflicts Scenario:

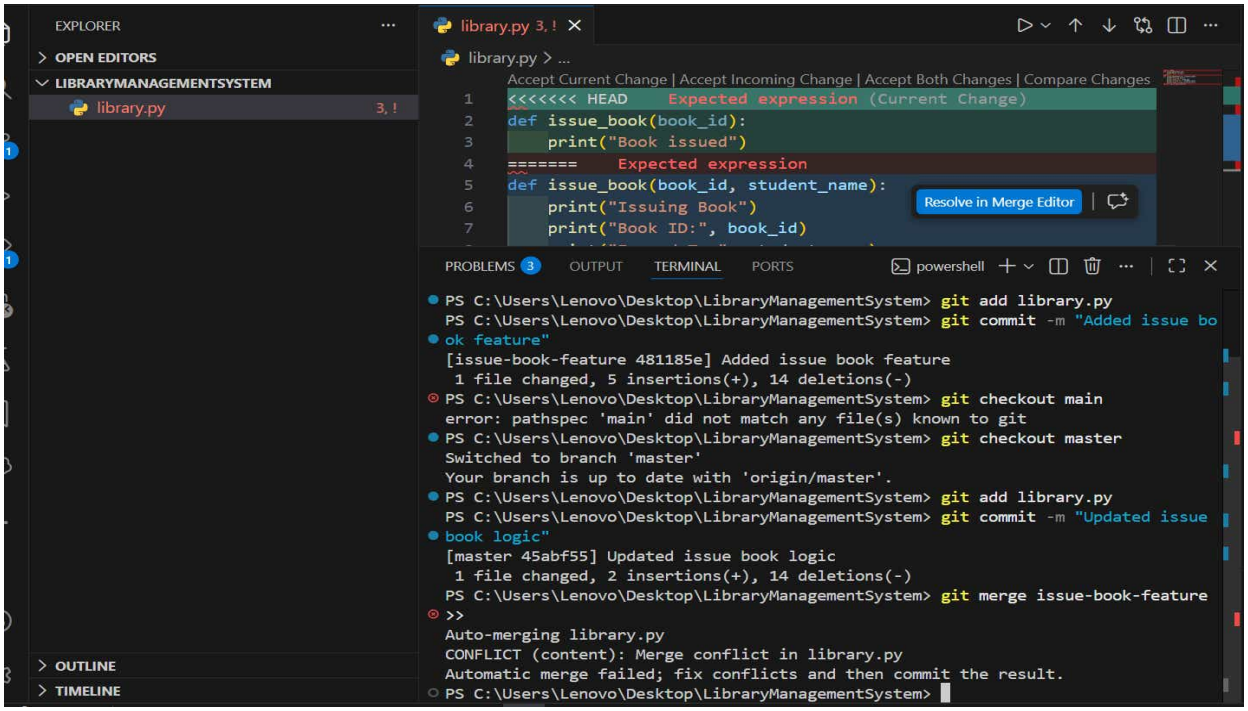
Two developers edited the same file.

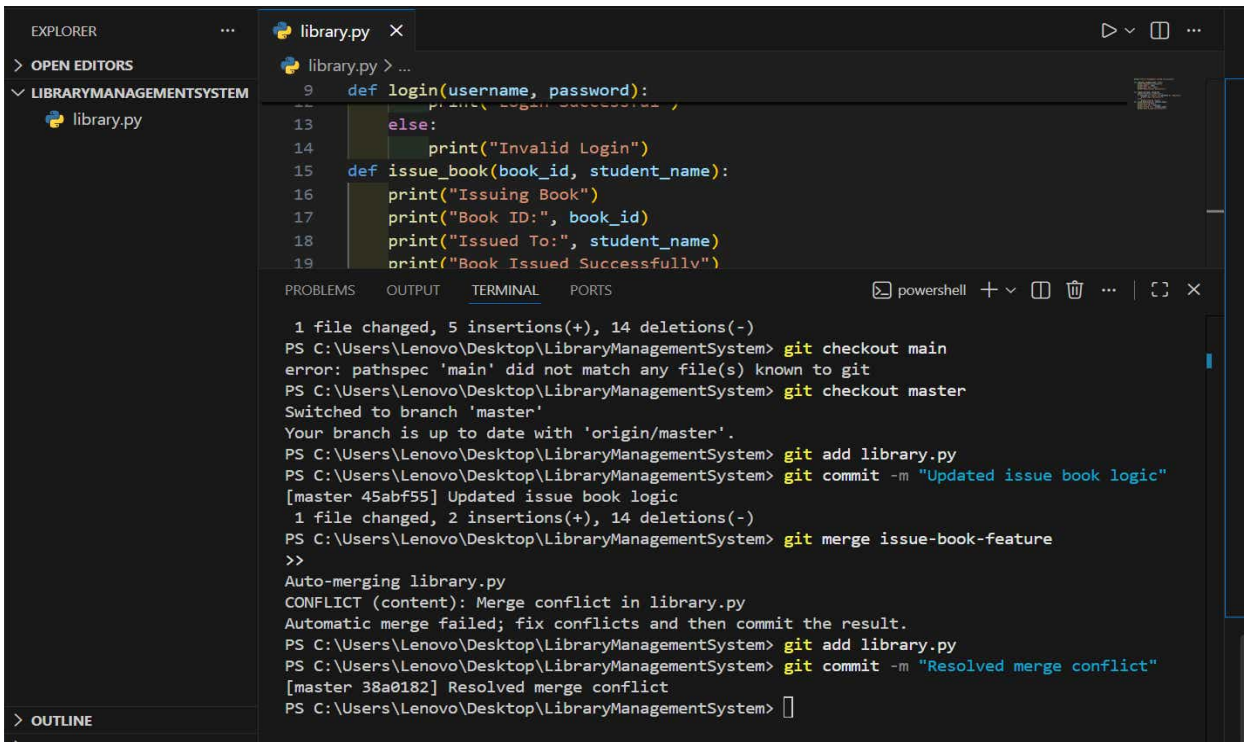
Tasks:

- Merge branches

- Resolve conflicts

- Commit resolved changes





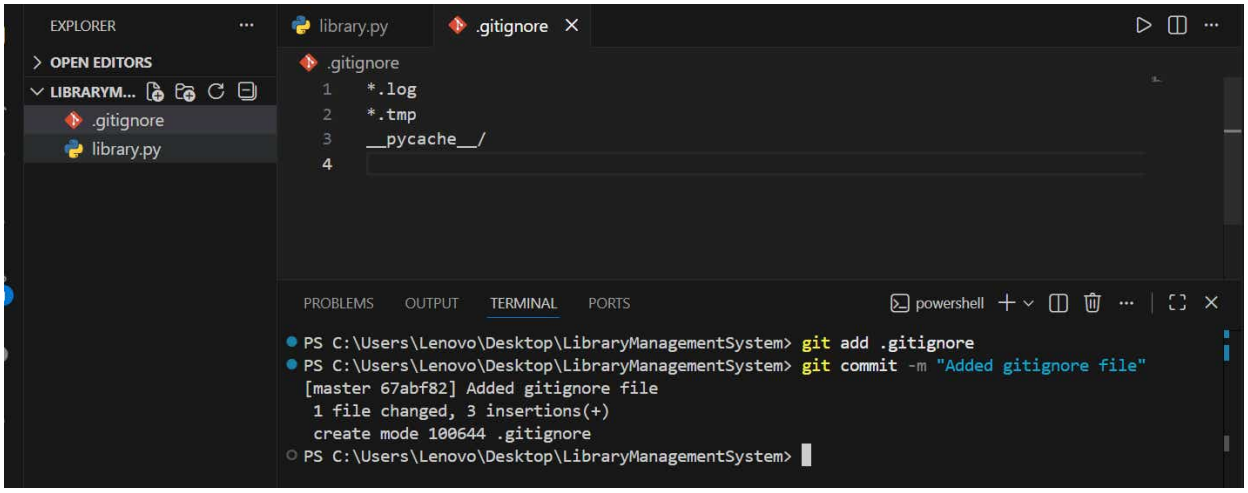
Case Study 8: Ignoring Unwanted Files

Scenario: Log and temporary files should not be tracked.

Tasks:

- Create .gitignore file

- Add ignore patterns



Case Study 9: Collaborating with Team Scenario:

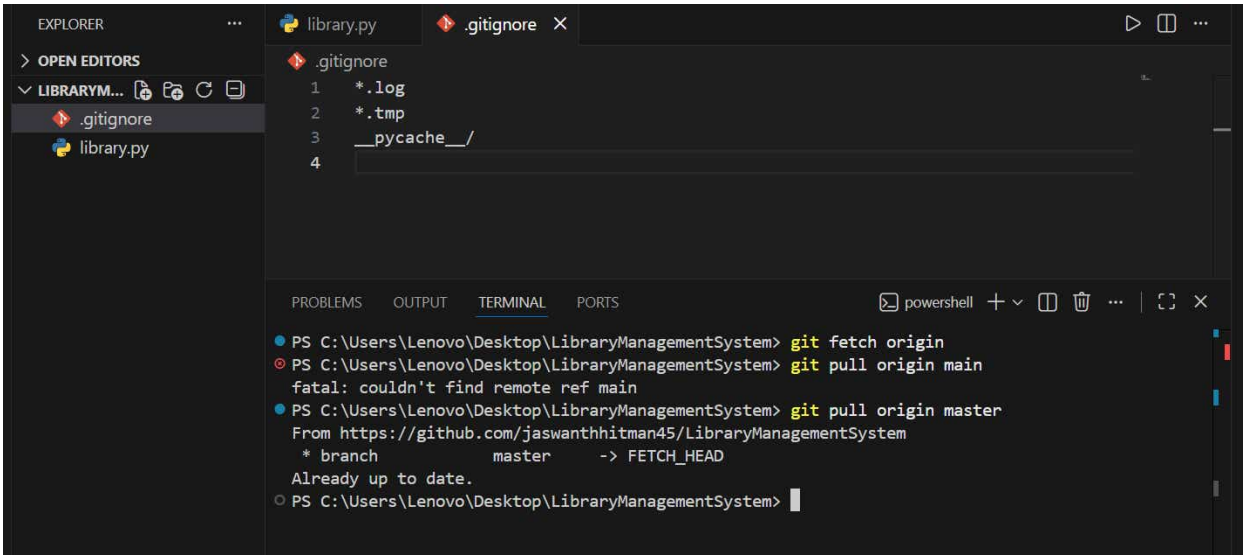
A teammate pushed new changes.

Tasks:

- Fetch latest changes

- Pull updates

- Resolve conflicts if any



Case Study 10: Final Project Release Scenario:

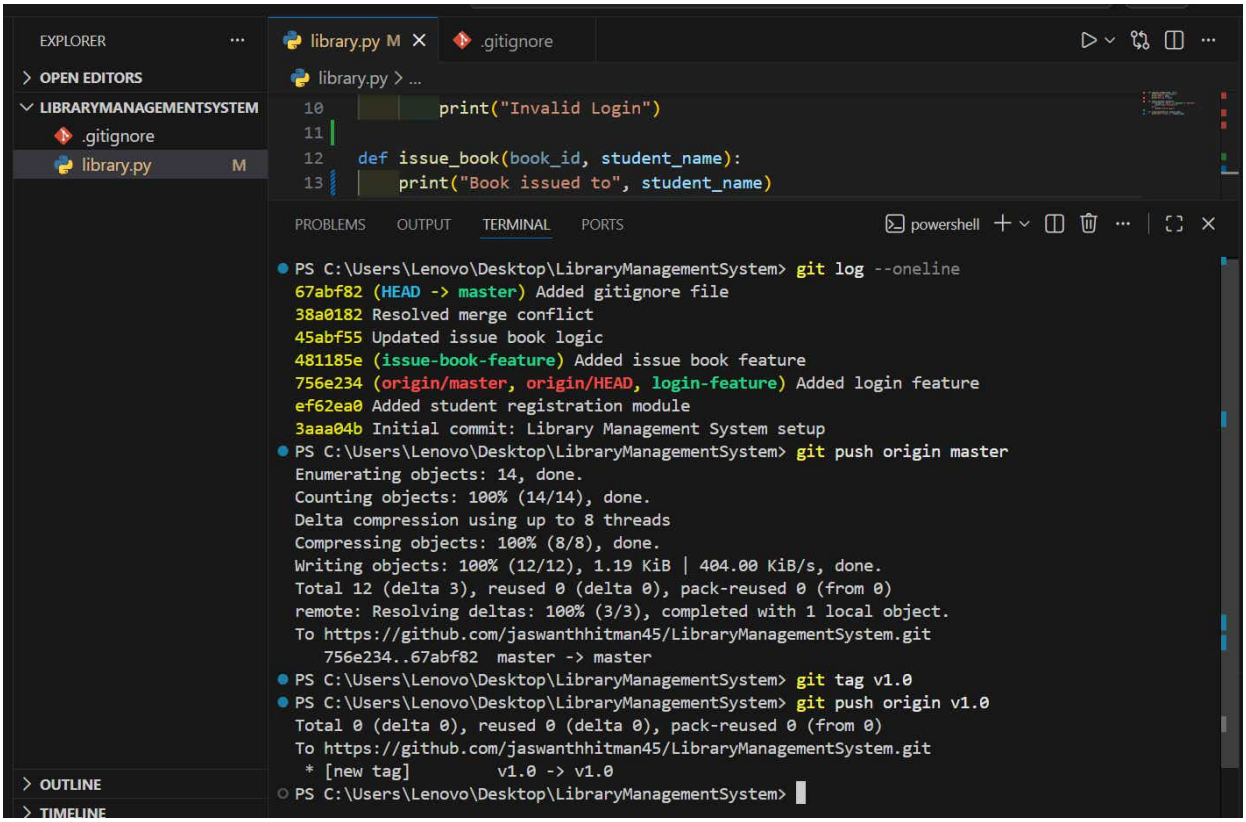
Project is ready for final submission.

Tasks:

- Review commit history

- Push final code

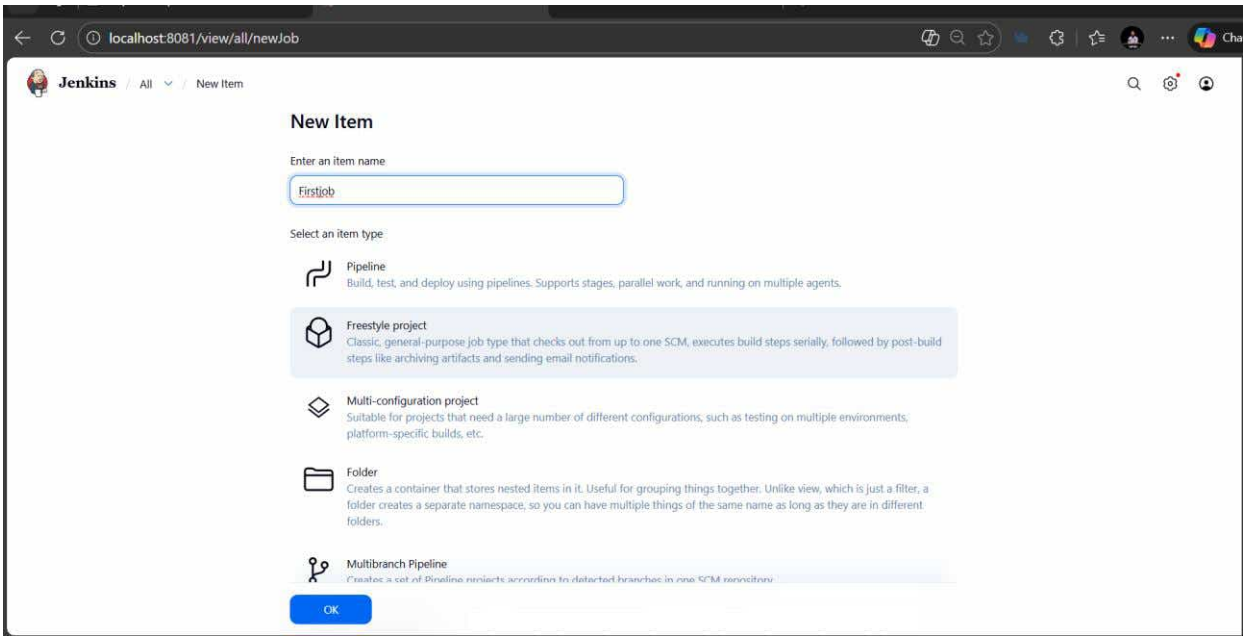
- Prepare release version



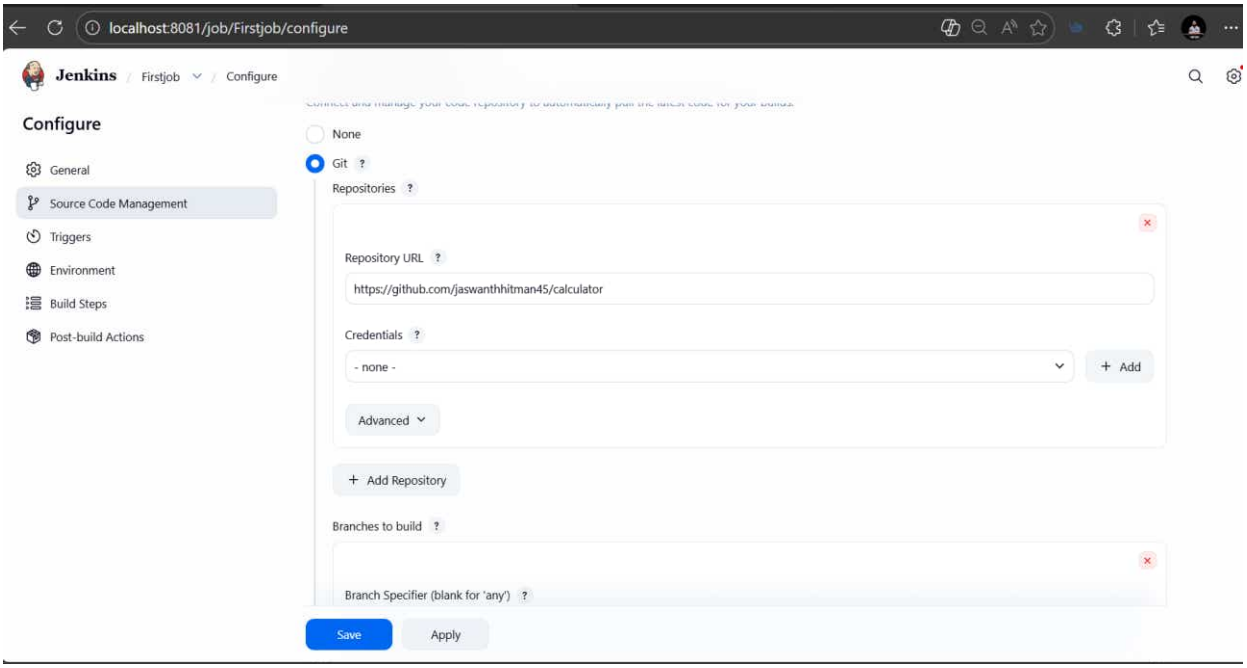
Jenkins

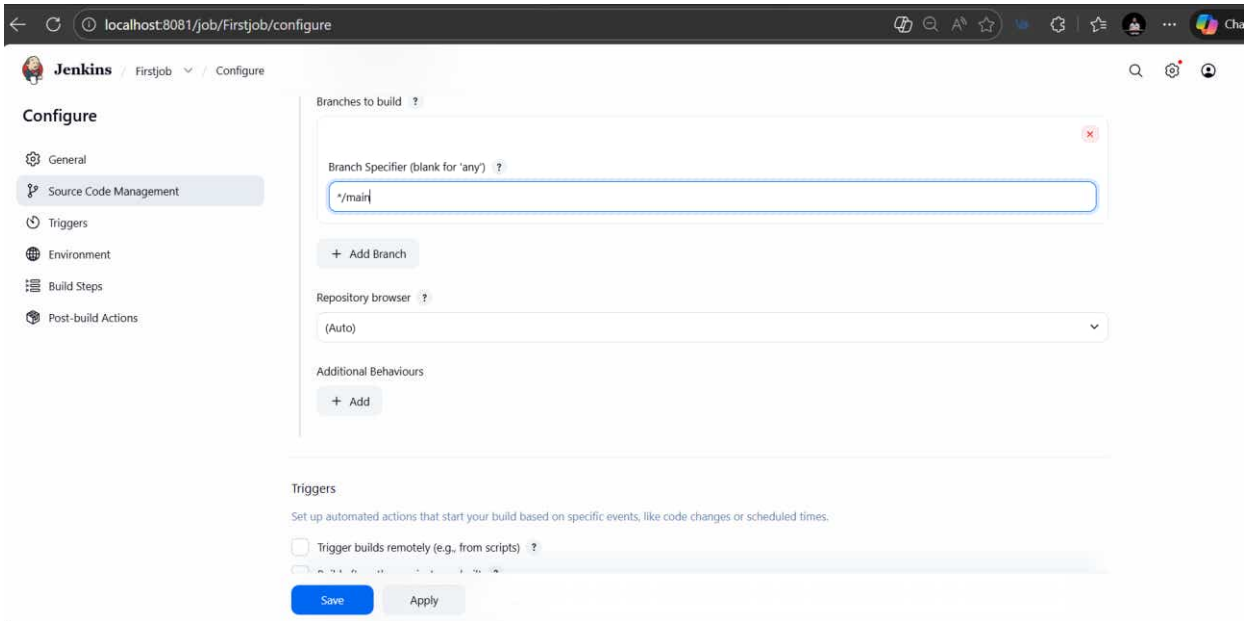
First job Creation:

Creating a New Freestyle Job:

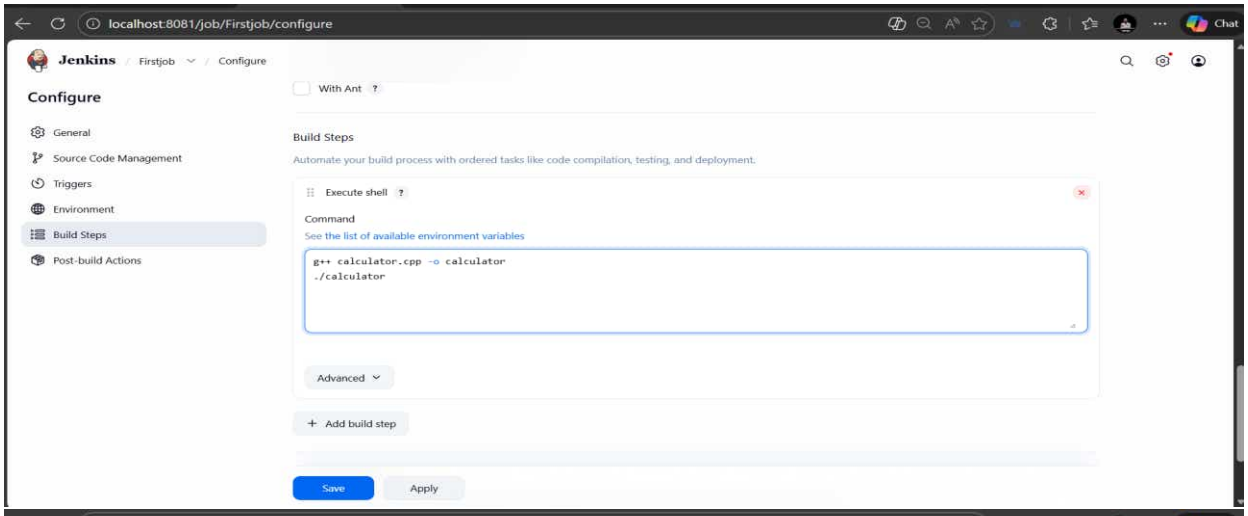


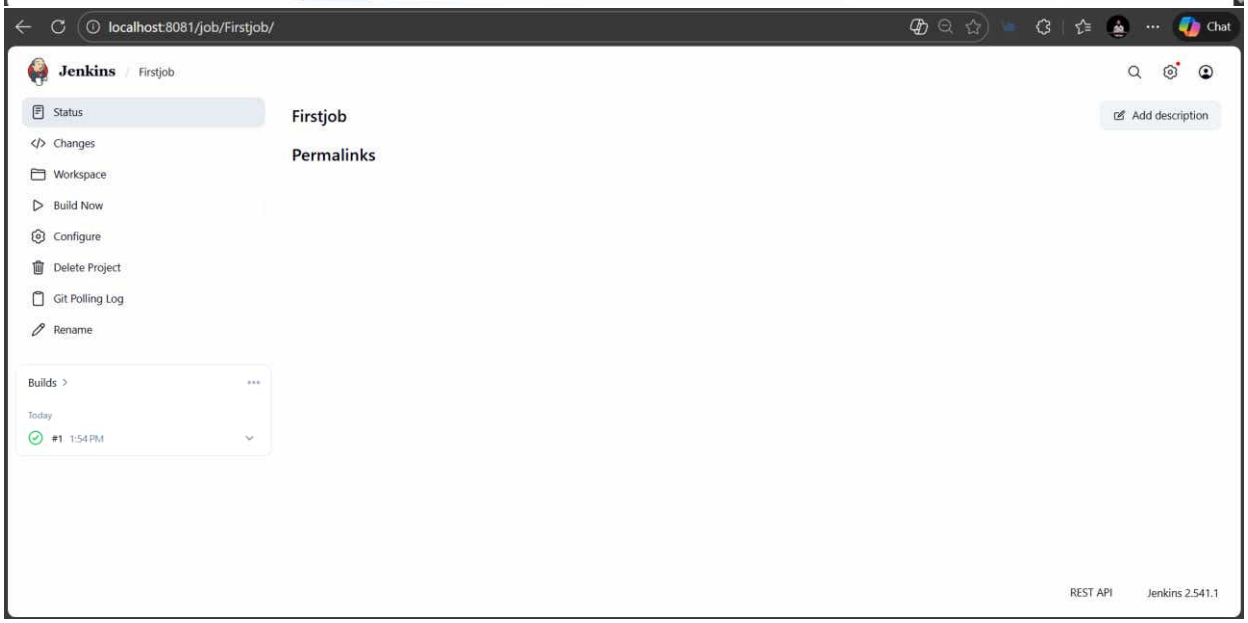
Configuring Git Repository in Jenkins Job:



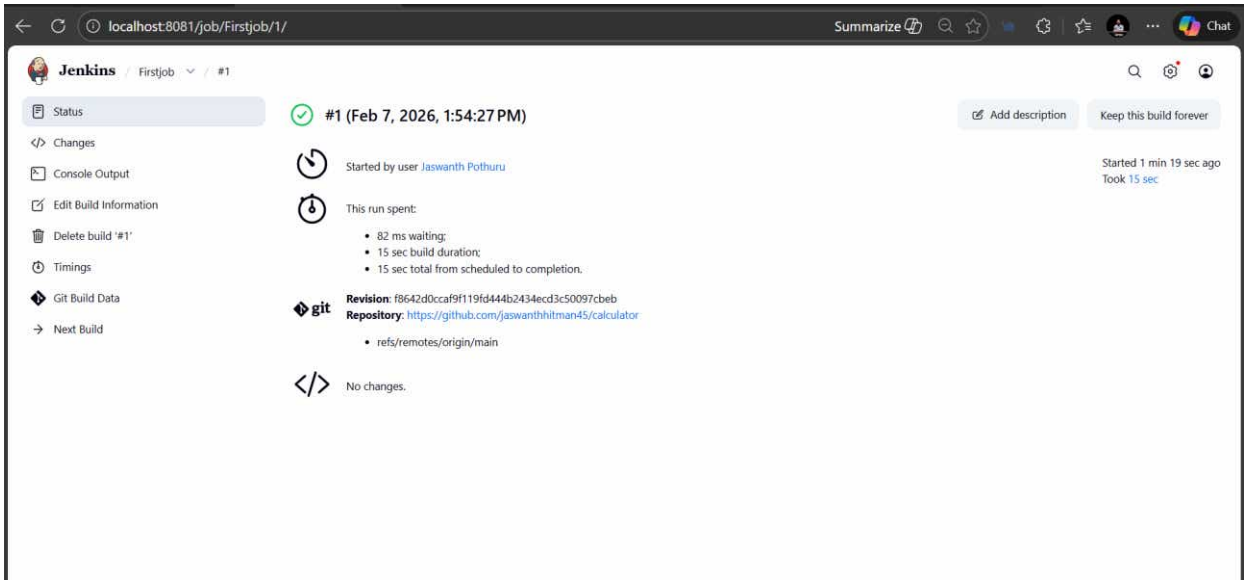


Adding Build Step to Execute Program:

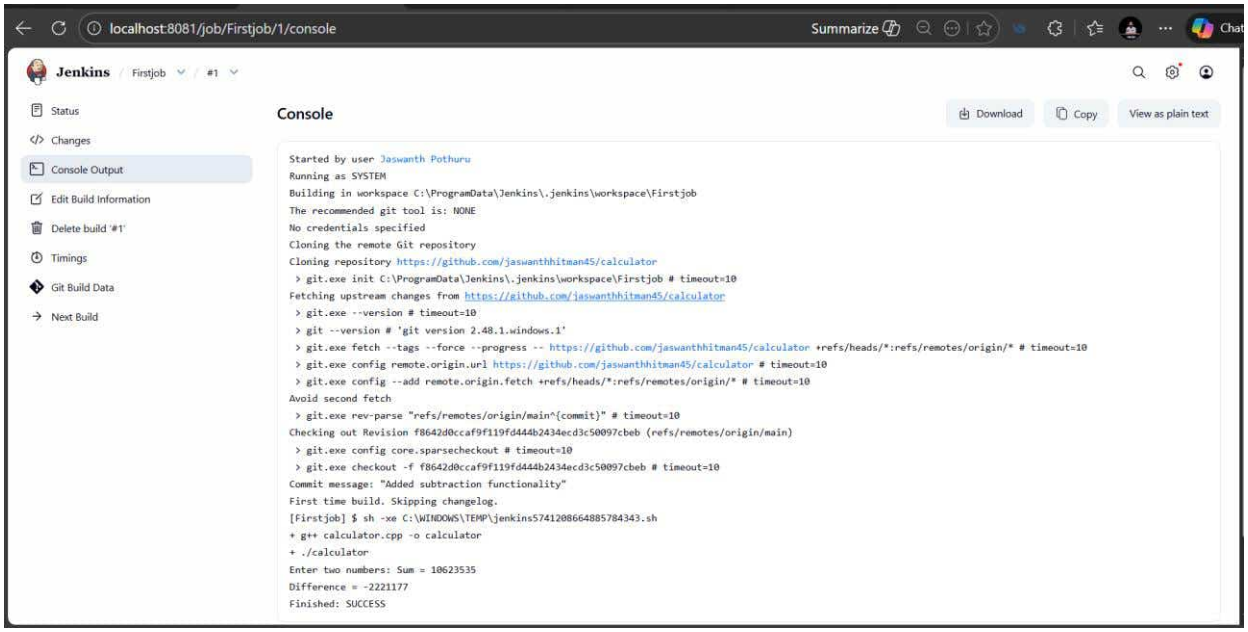




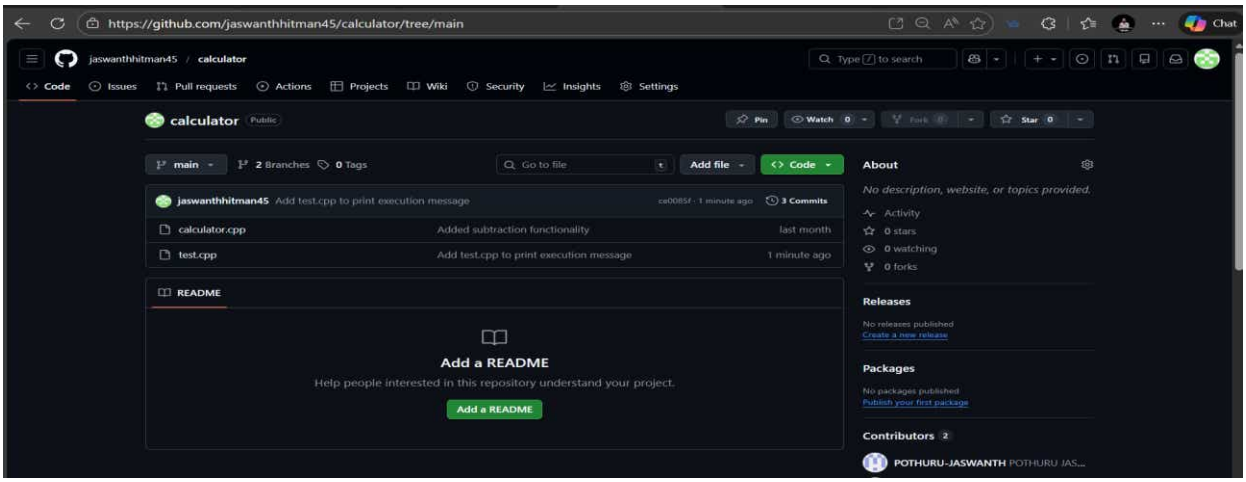
Building the Jenkins Job:

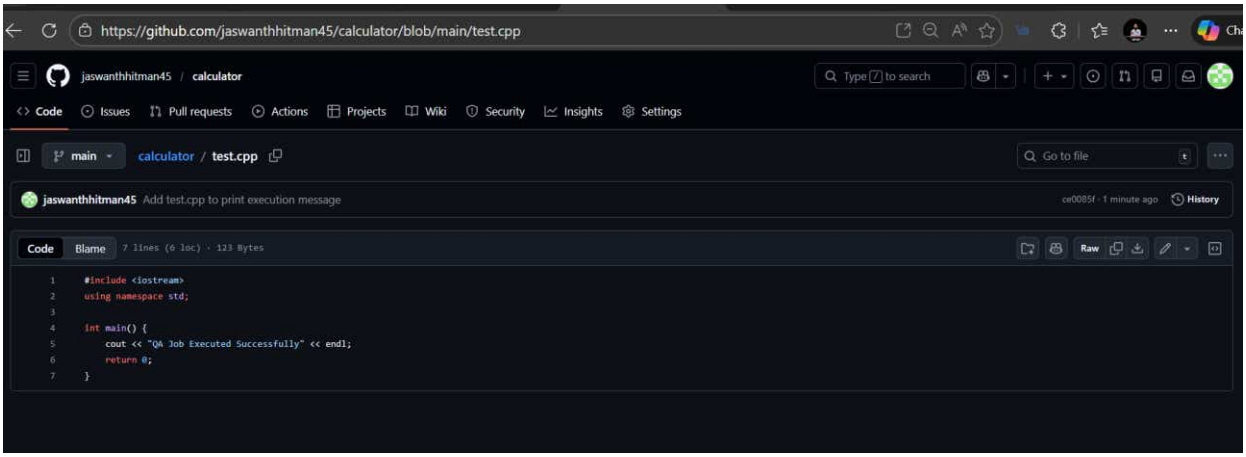


Console Output of Jenkins Job:

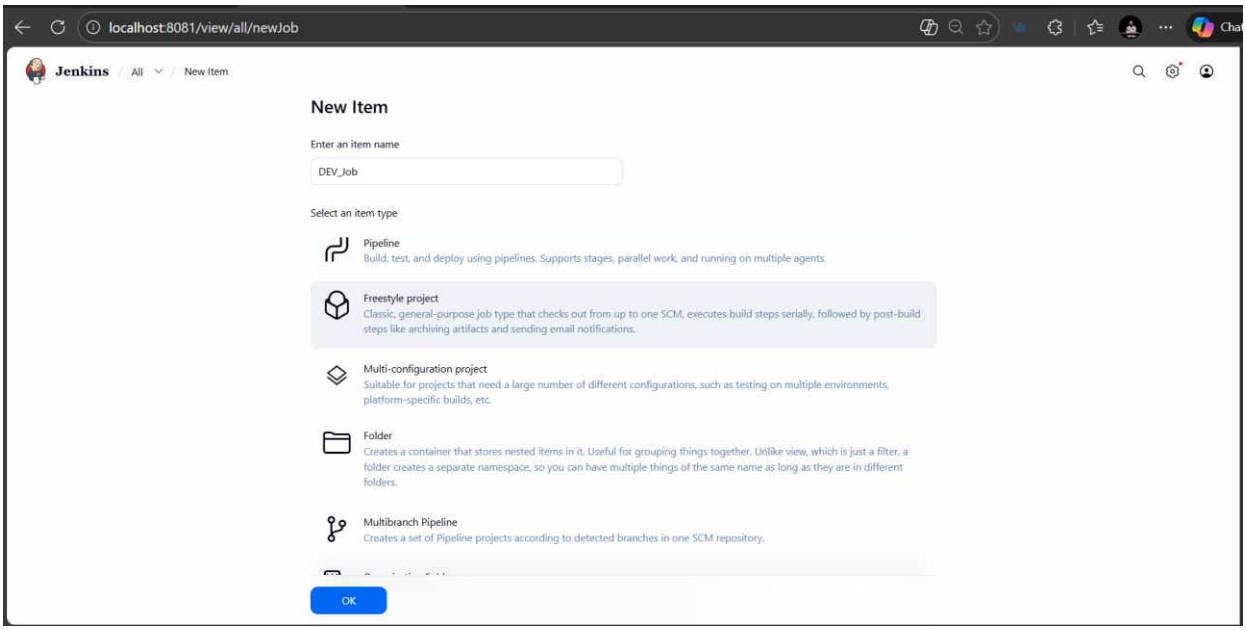


PART-B: CI/CD PIPELINE USING TWO FREESTYLE JOBS

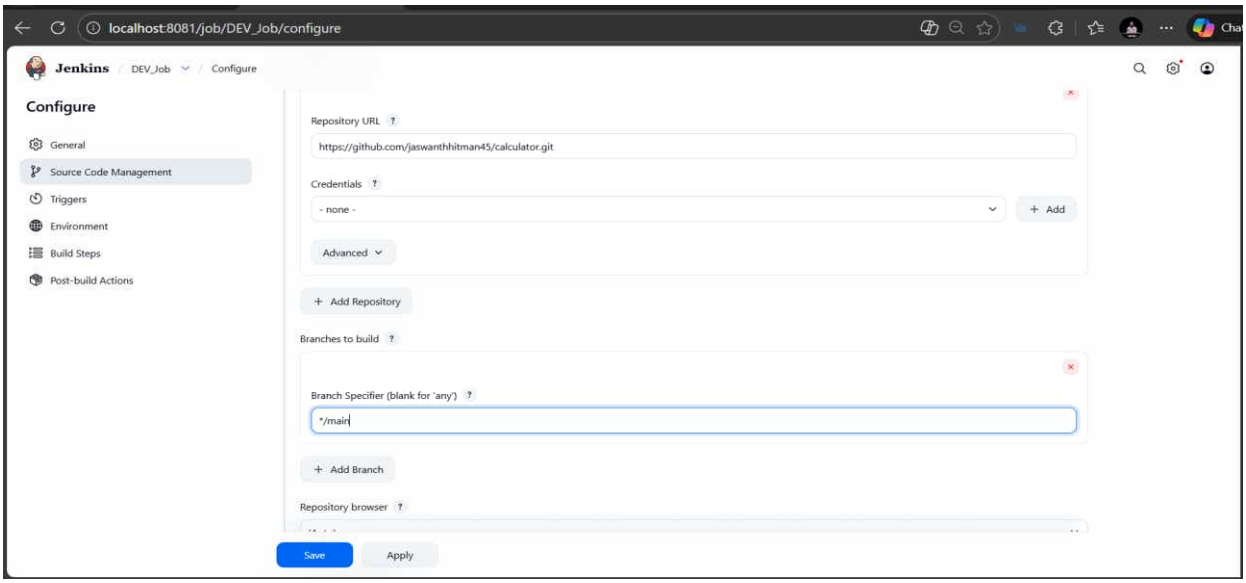




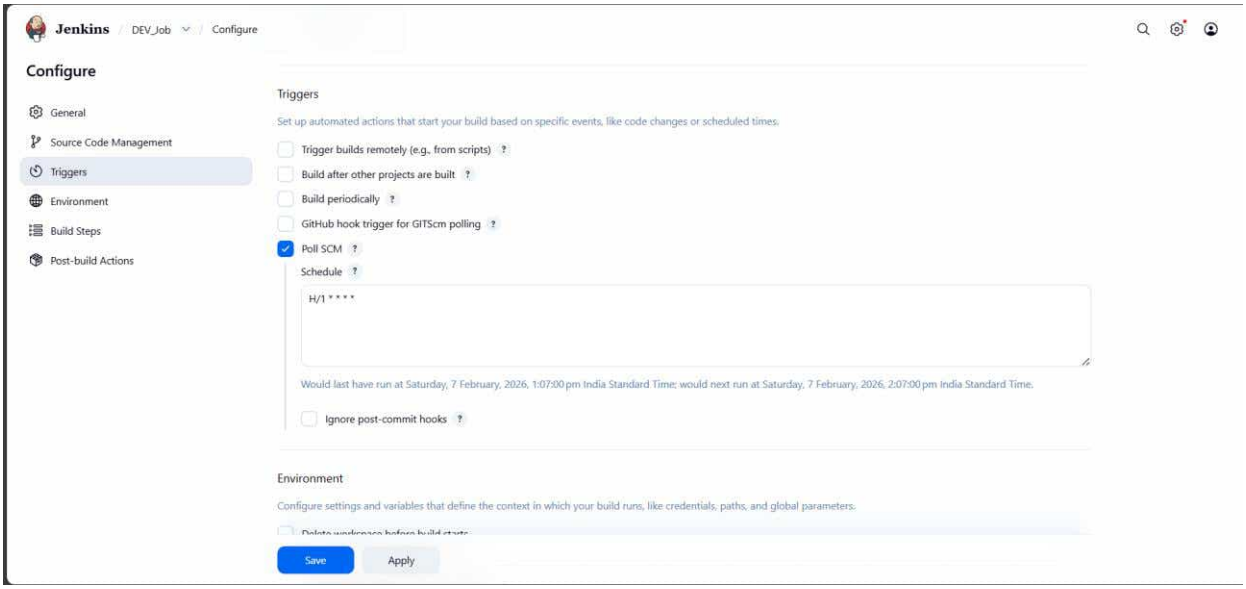
Creating DEV Job in Jenkins:



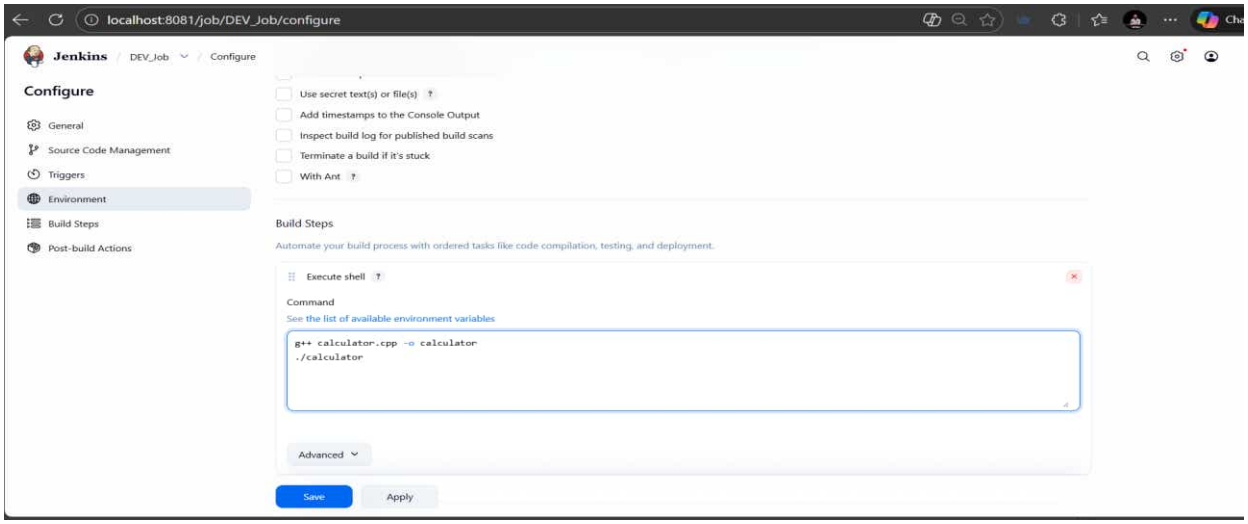
Configuring Git Repository in DEV Job:

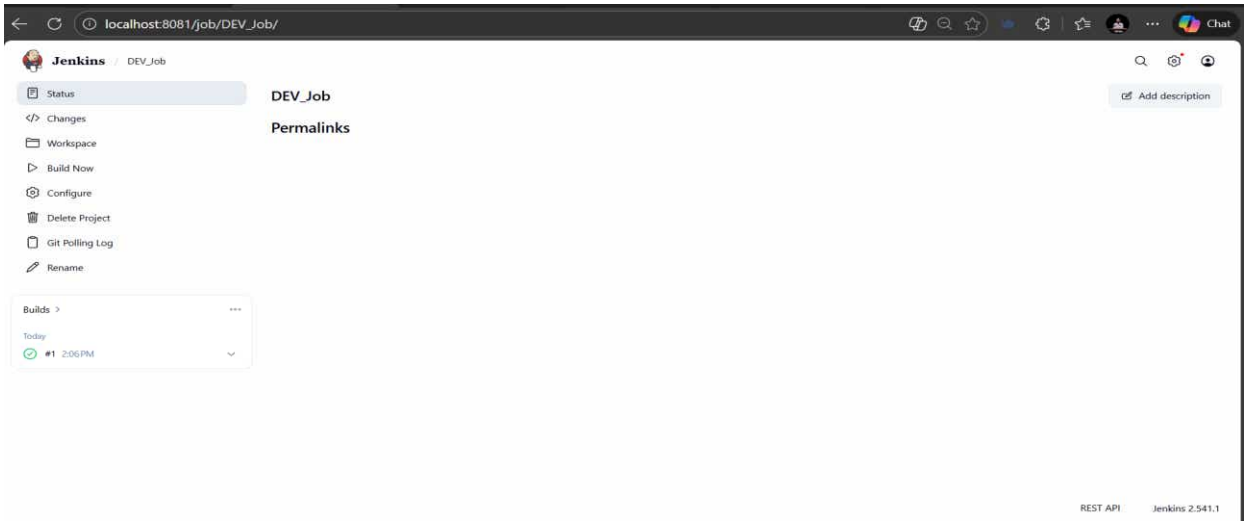


Enabling Poll SCM in DEV Job to check Git for changes every minute.

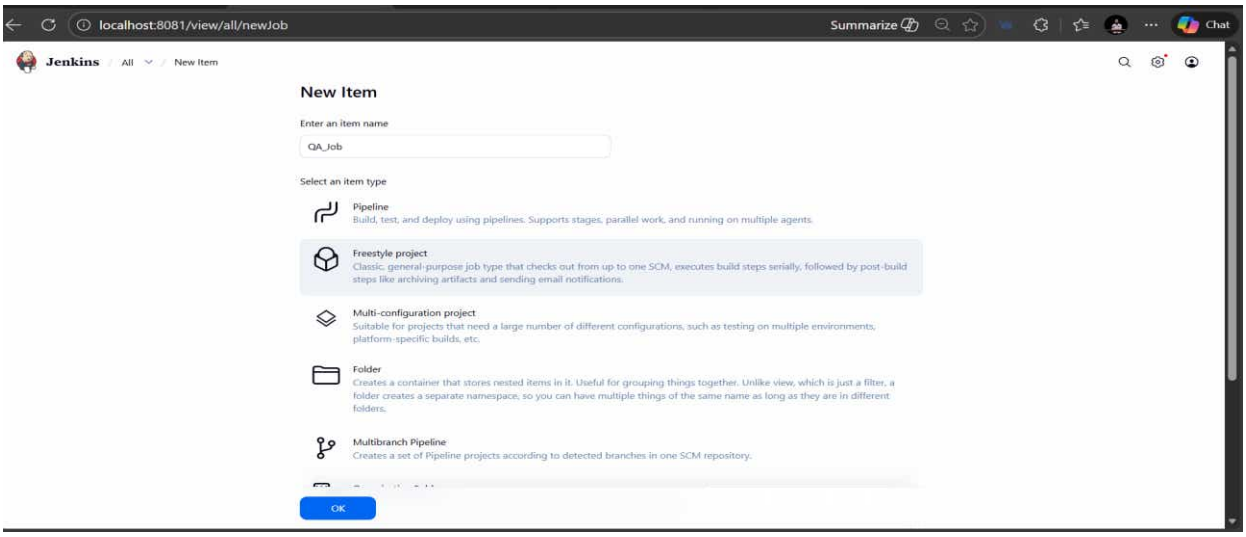


DEV Job Build Step Configuration

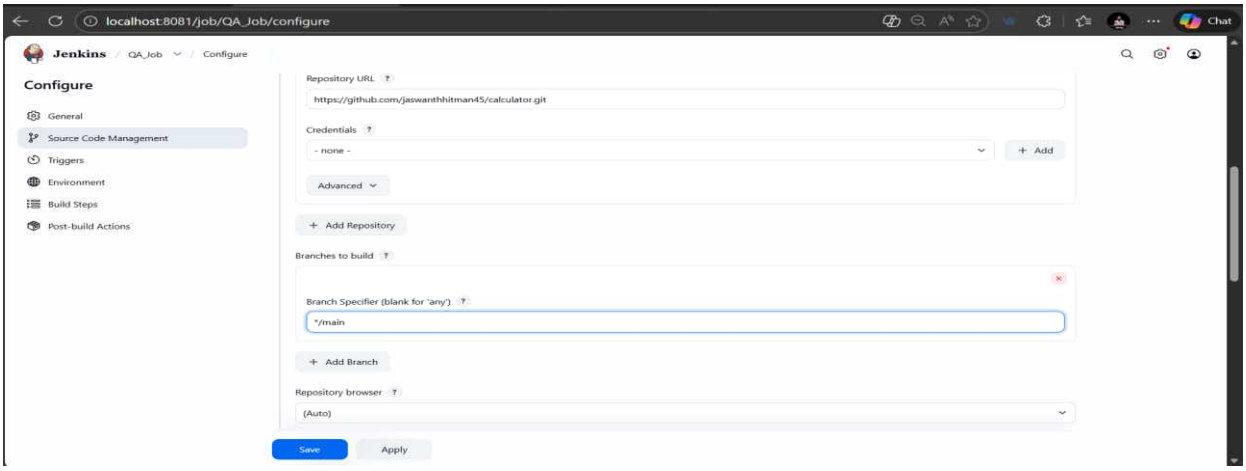




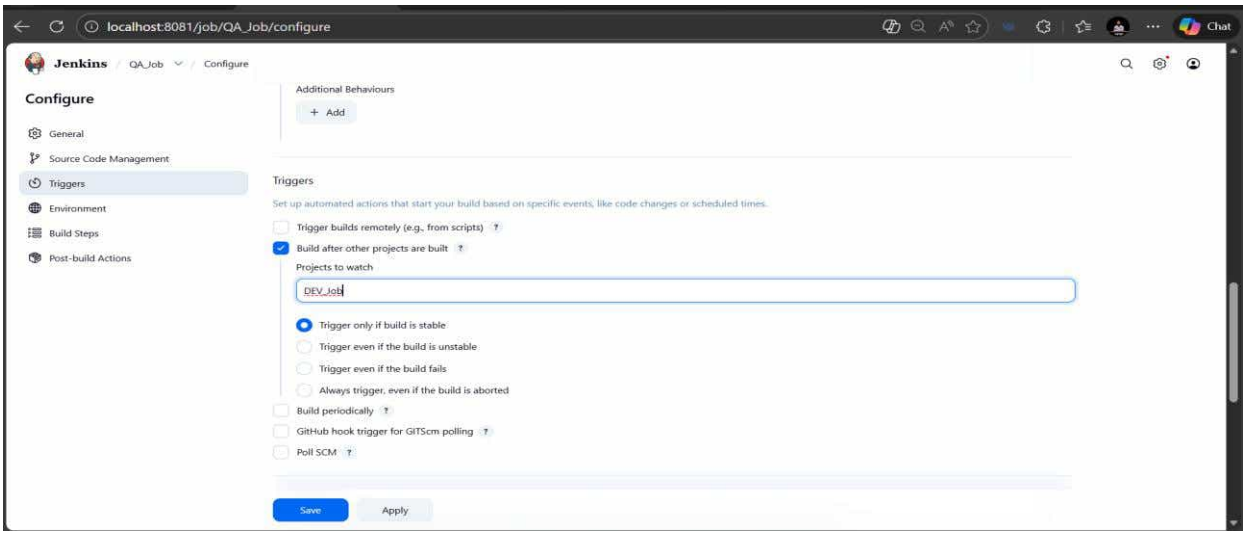
Created a new Freestyle job named QA_Job in Jenkins.



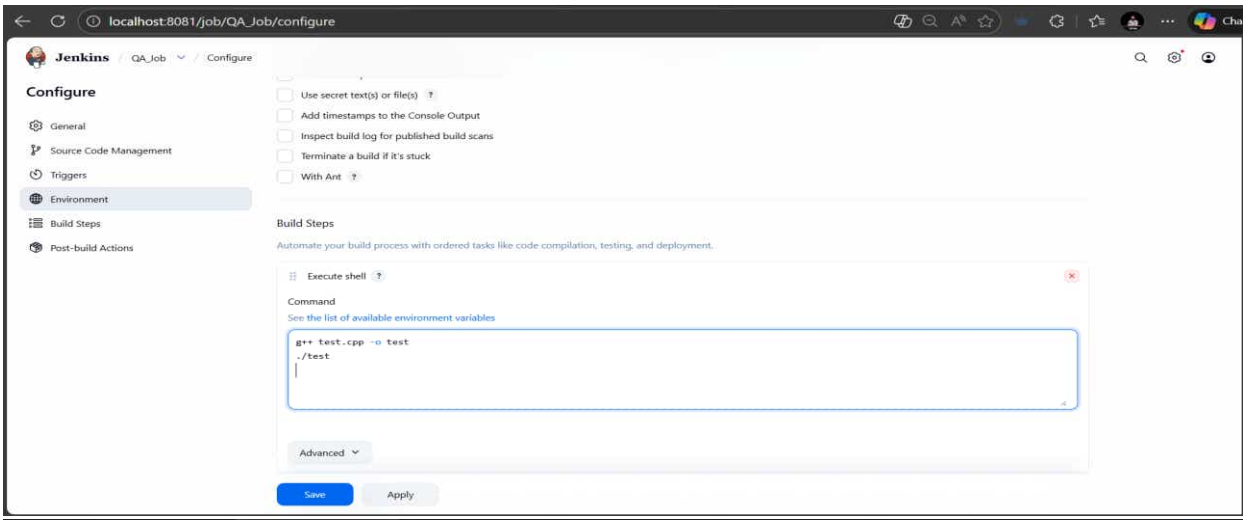
Configured same Git repository and branch in QA_Job.

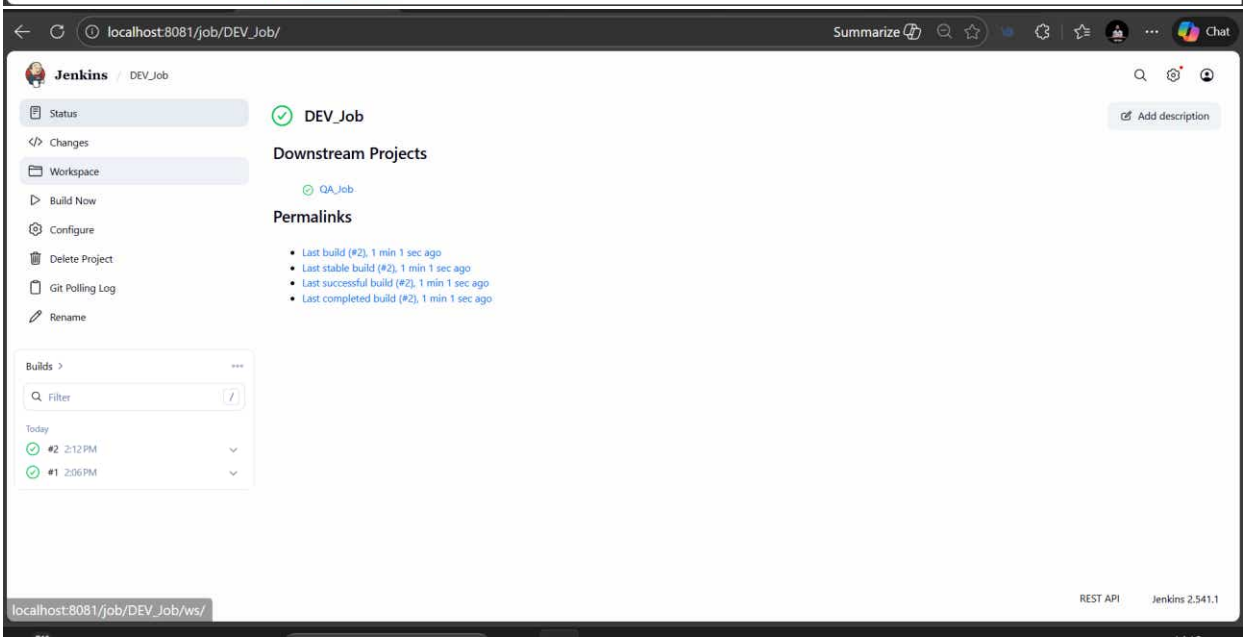


Configured QA_Job to run automatically after successful completion of DEV_Job:

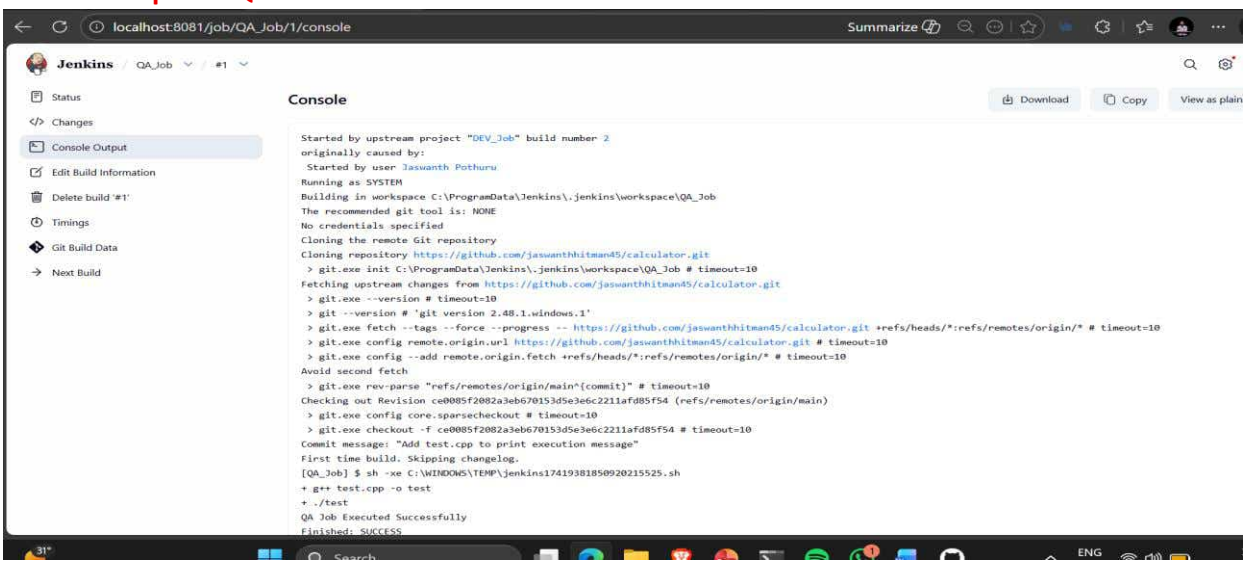


QA Job Build Step Configuration:

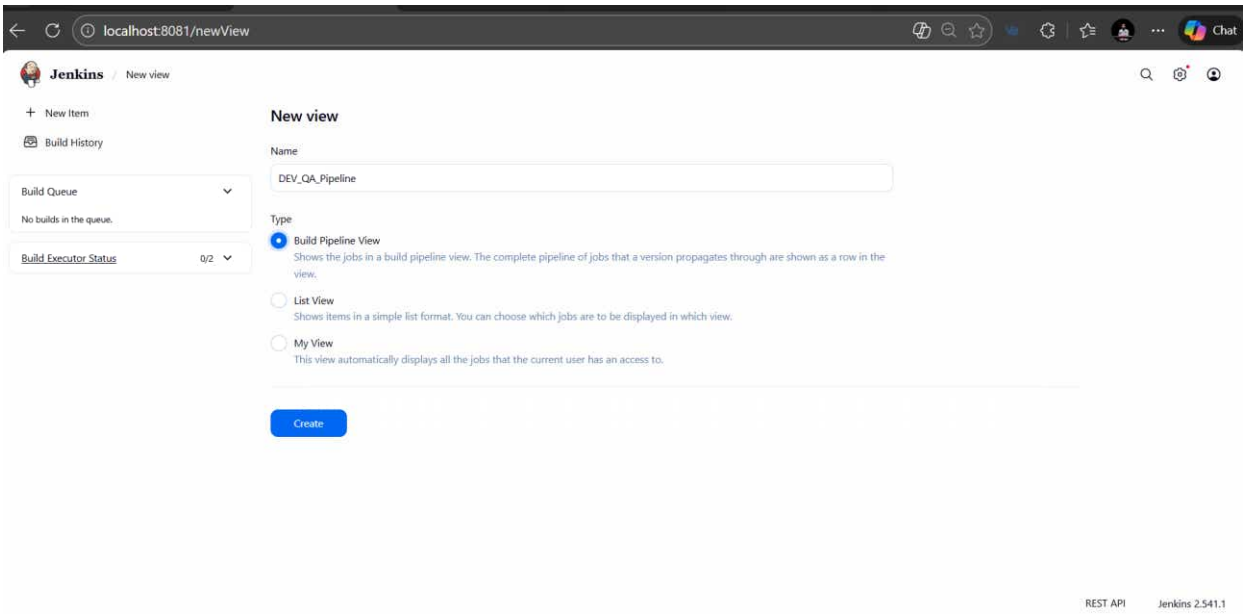




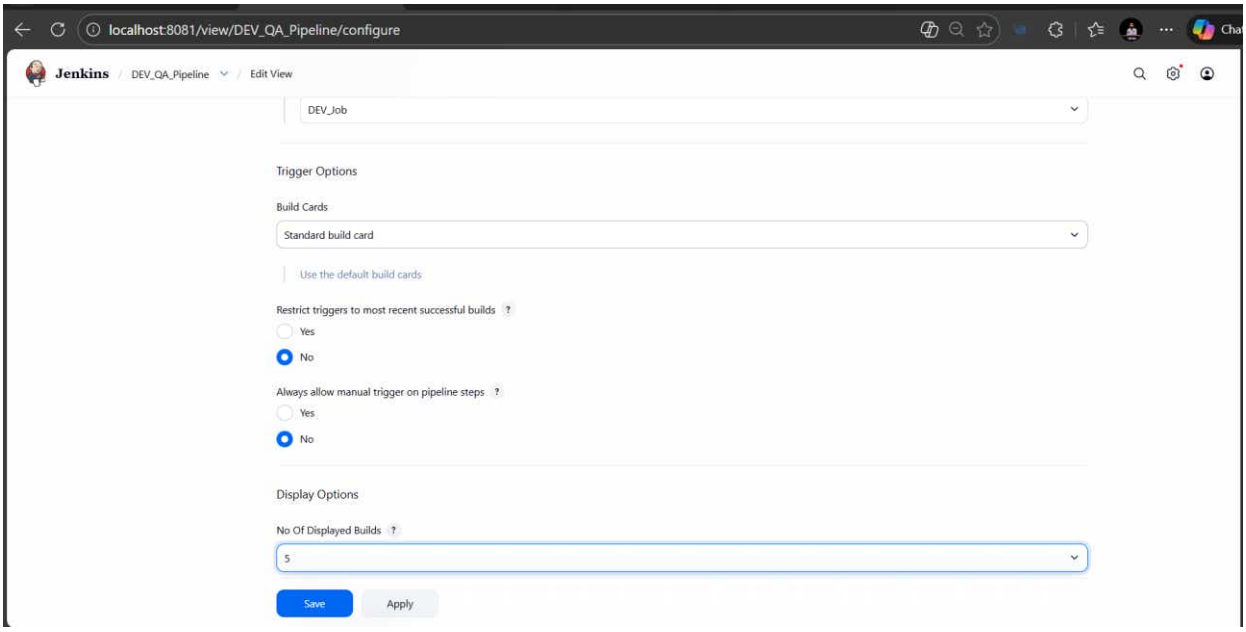
Console Output of QA Job:



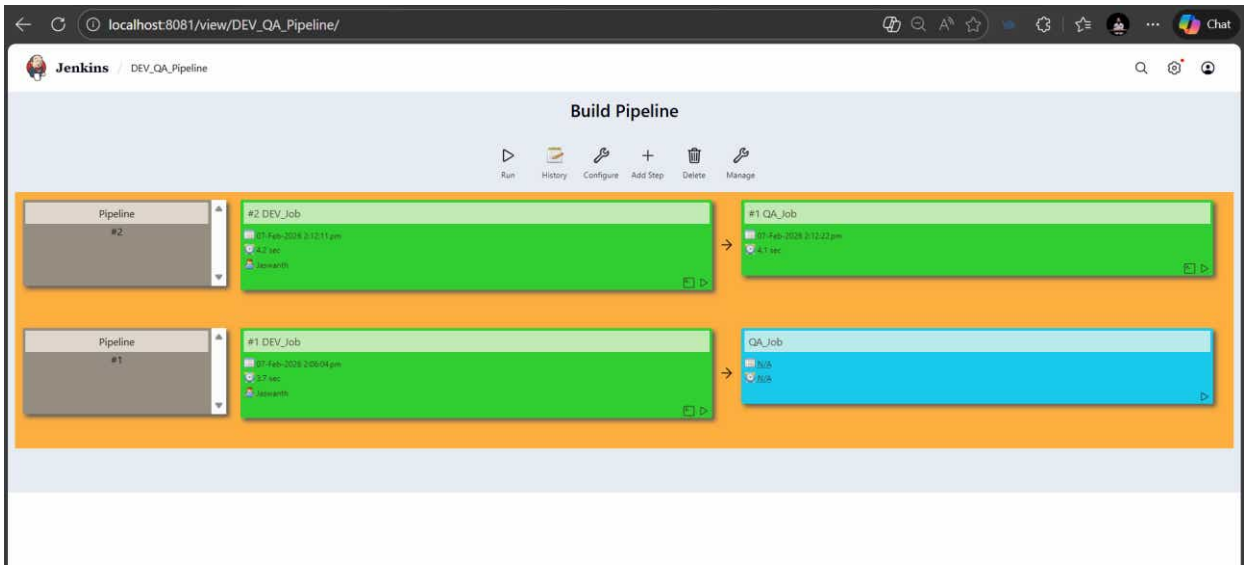
Created a Build Pipeline View named DEV_QA_Pipeline.

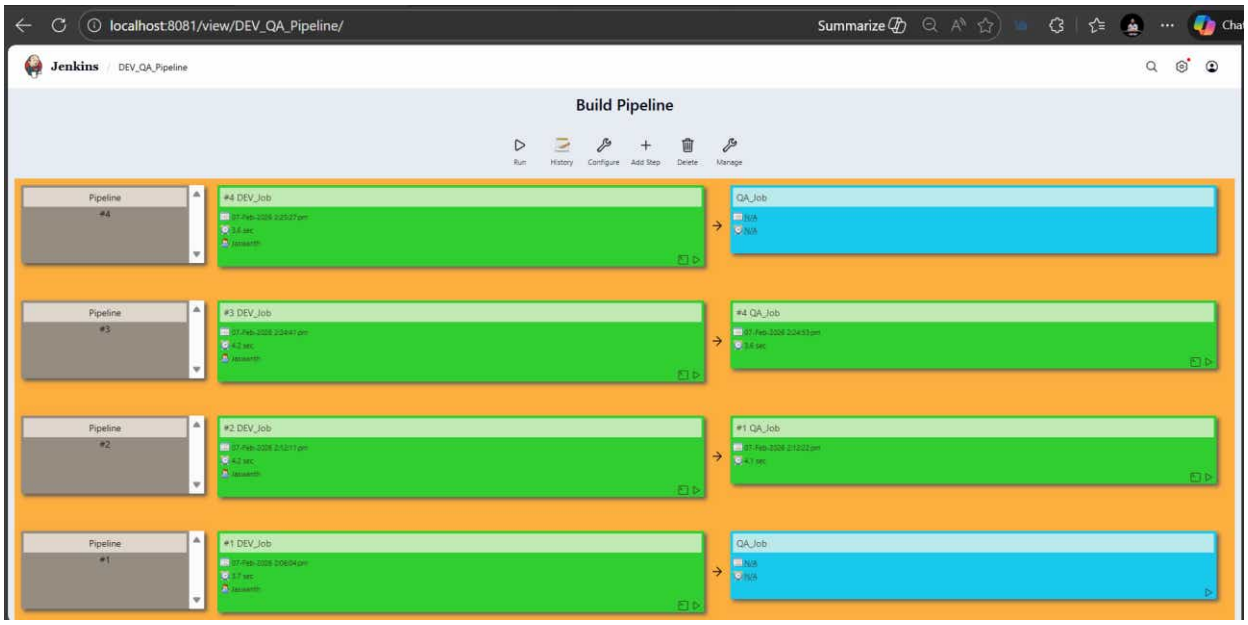


Set DEV_Job as the initial job in the pipeline view and limited display to 5 builds.



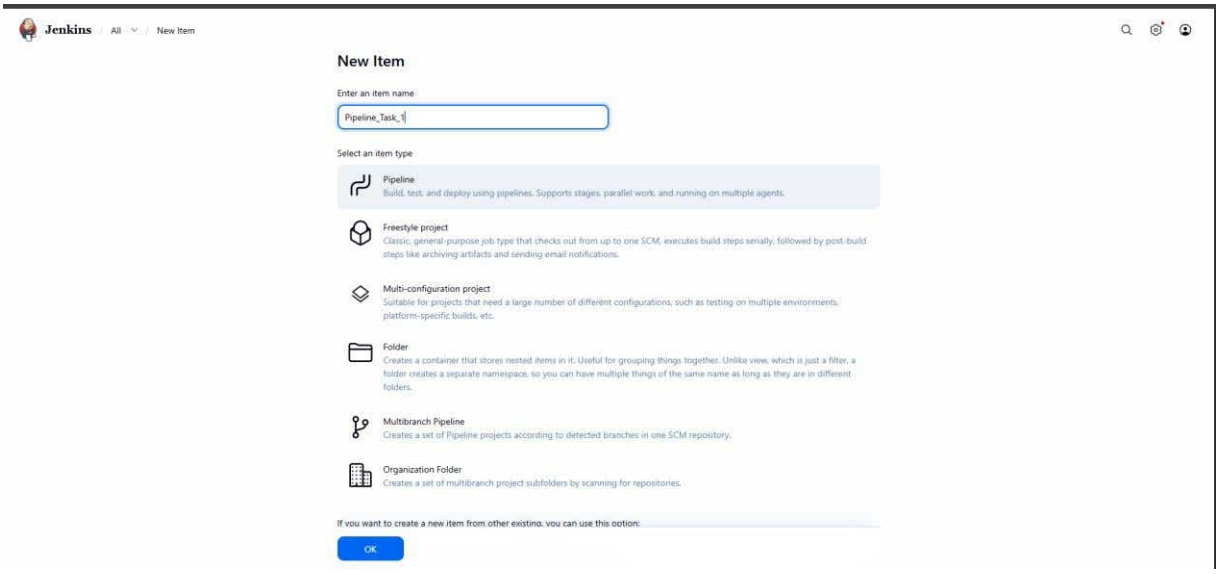
Executed the pipeline. DEV_Job ran first and upon successful completion, QA_Job ran automatically.



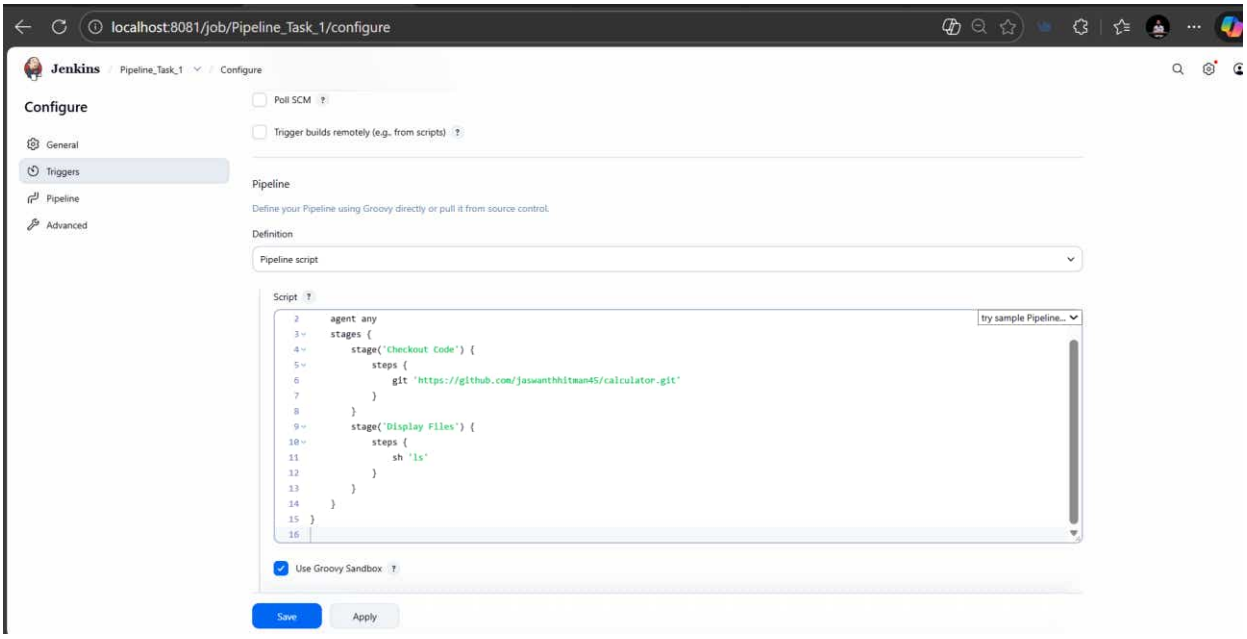


PIPELINE JOB (DIRECT PIPELINE SCRIPT IN JENKINS UI):

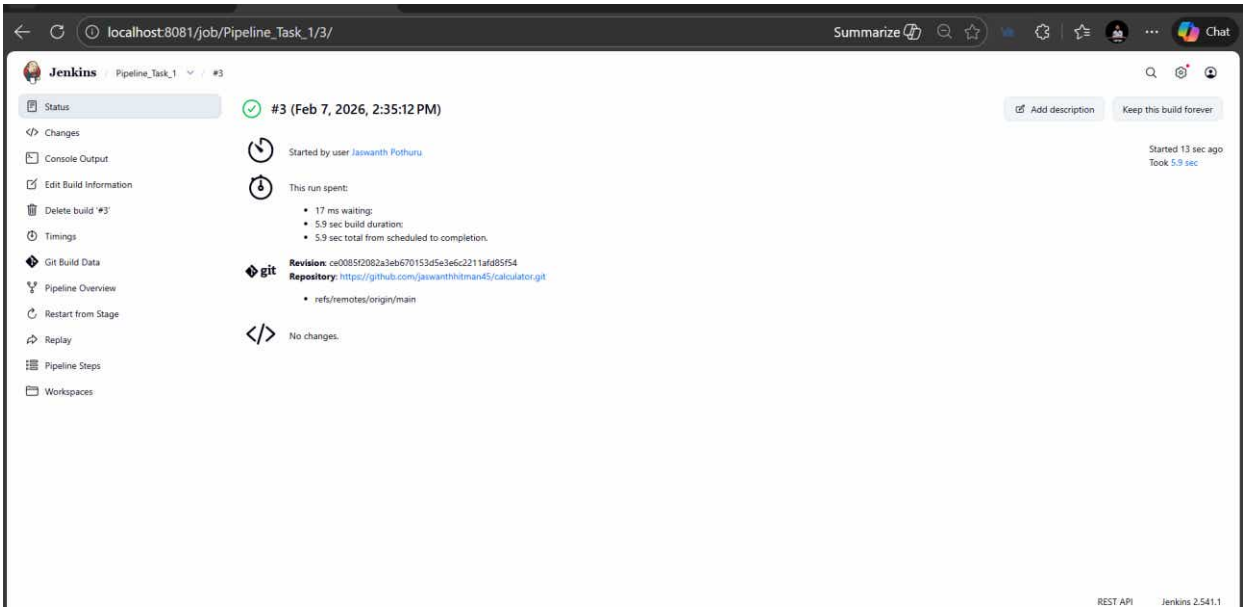
Creating Pipeline_Task_1 Job

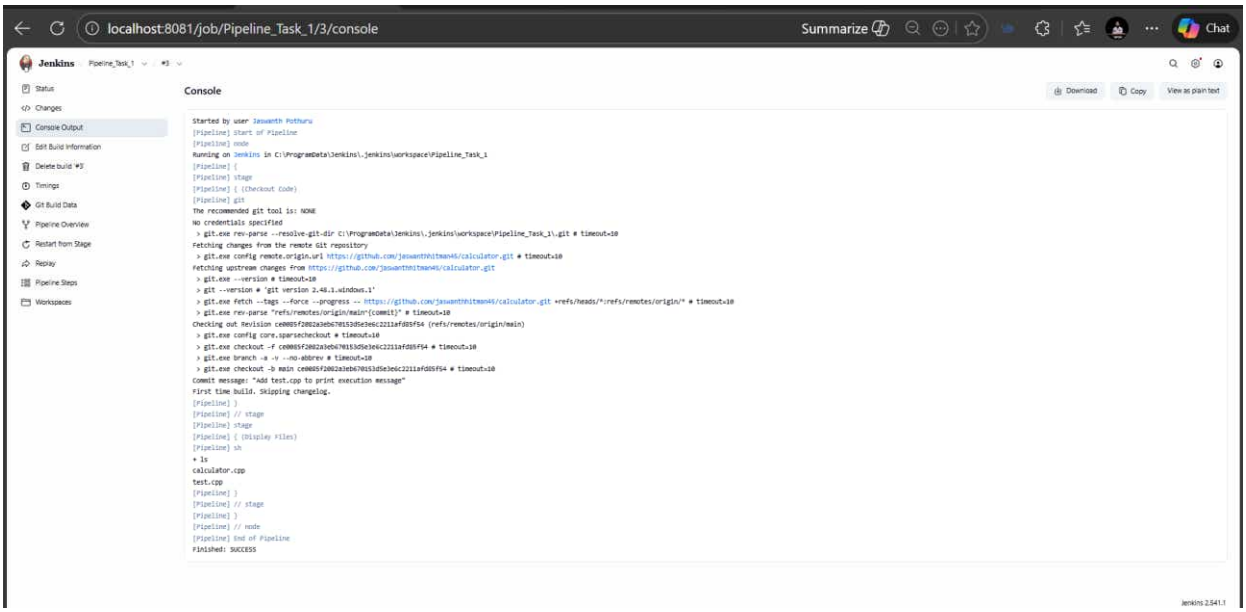


Adding Declarative Pipeline Script:

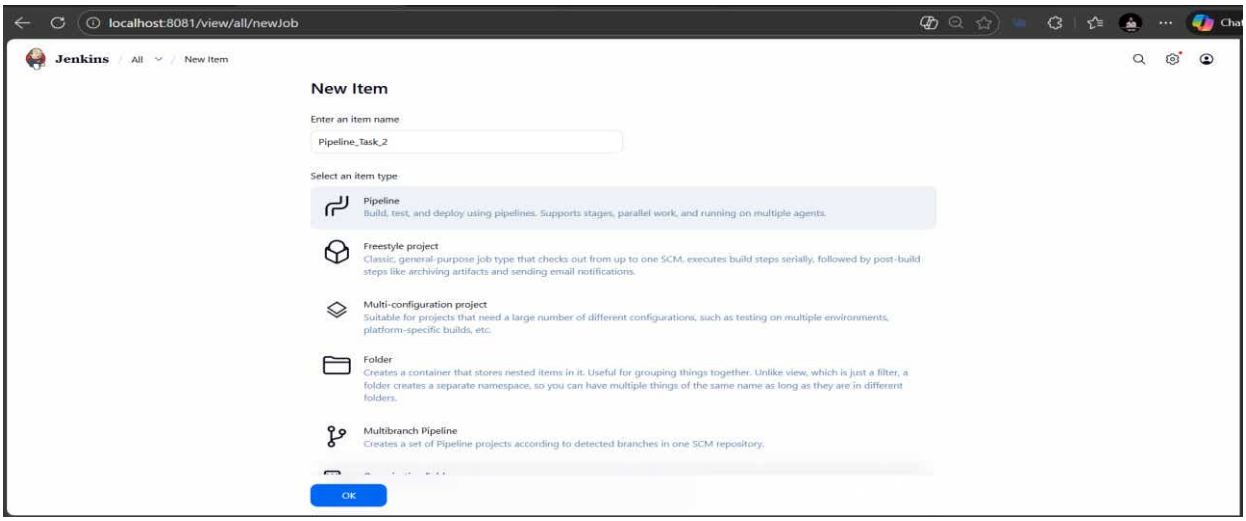


Executed Pipeline_Task_1 successfully and verified output in console.

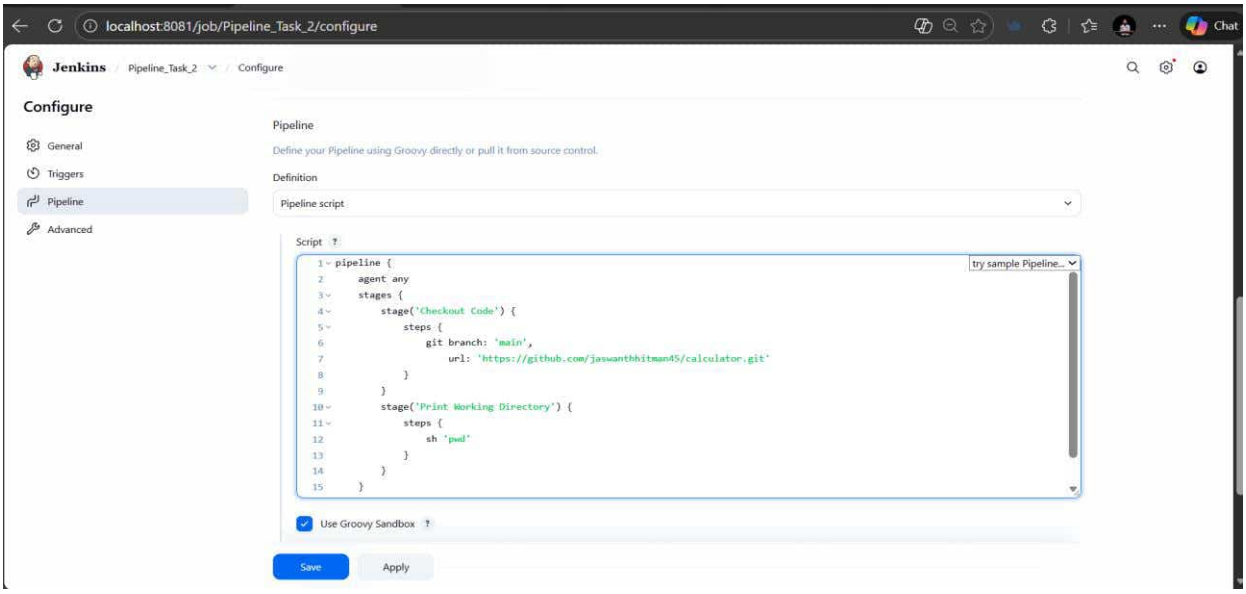


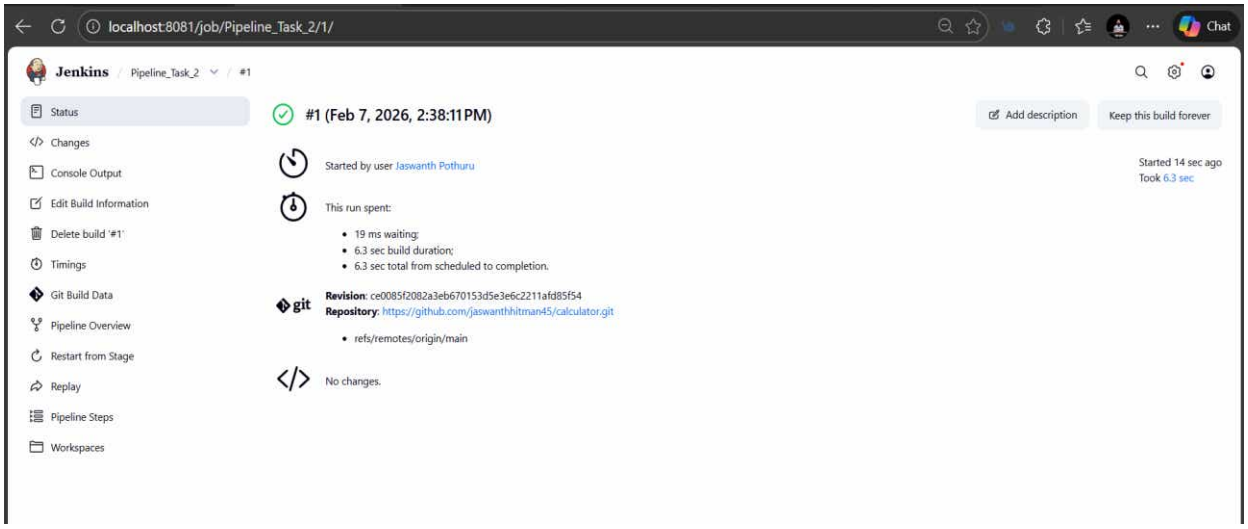


Created Pipeline_Task_2 to checkout code and print current working directory.

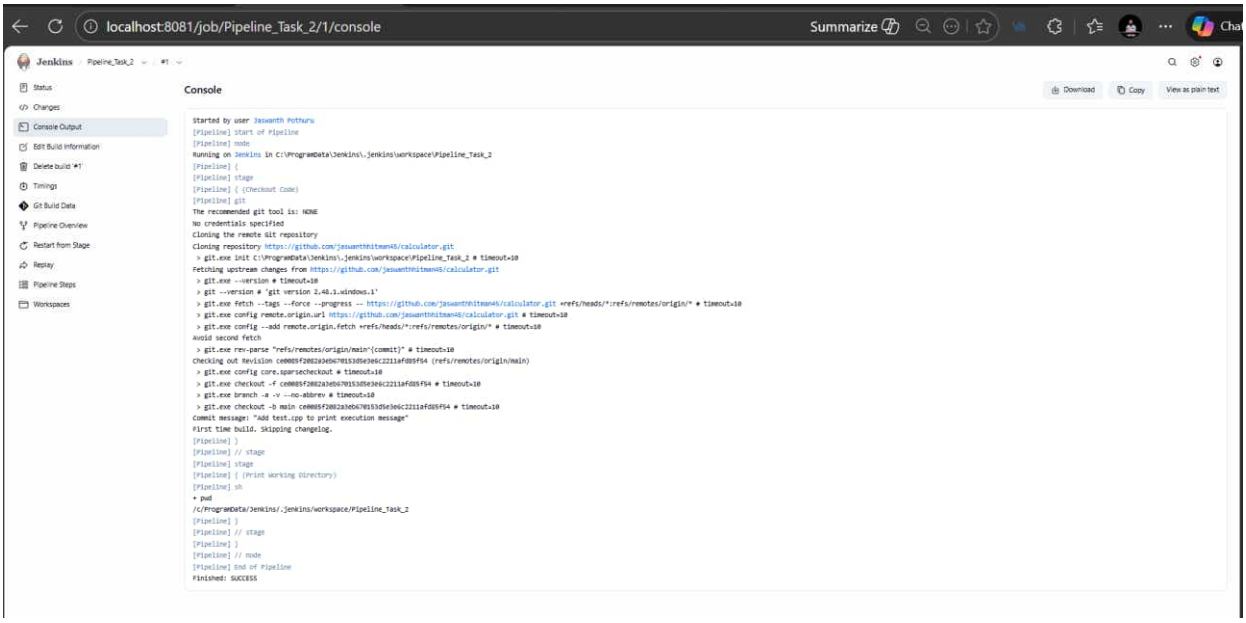


Adding Pipeline Script for Working Directory:

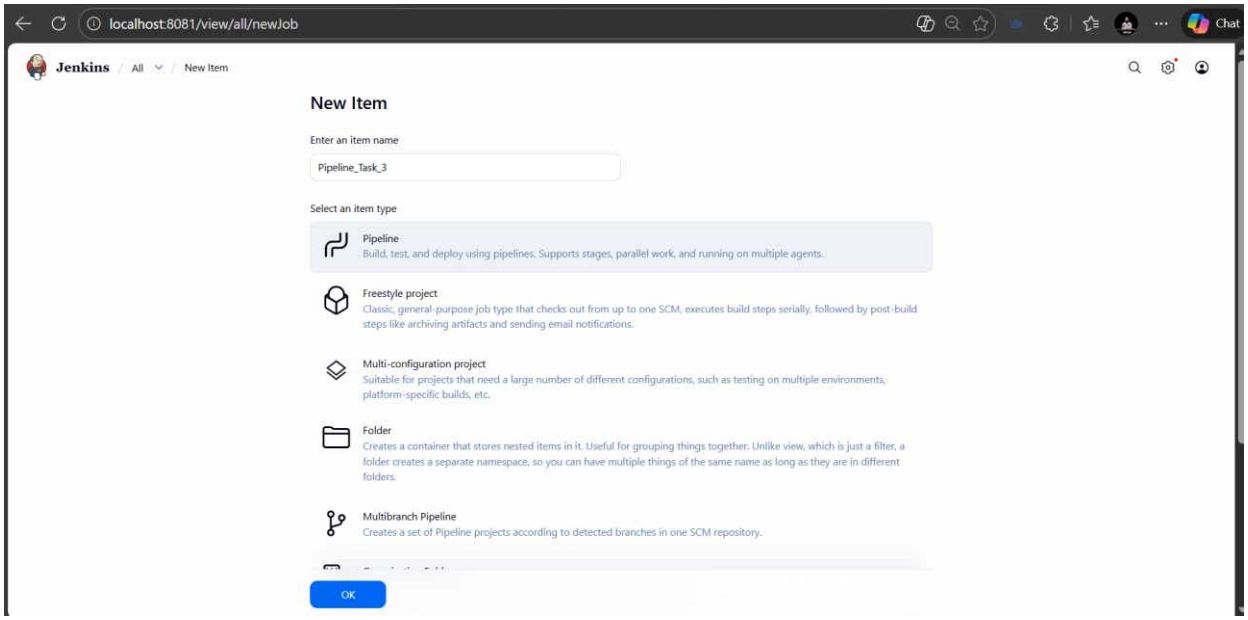




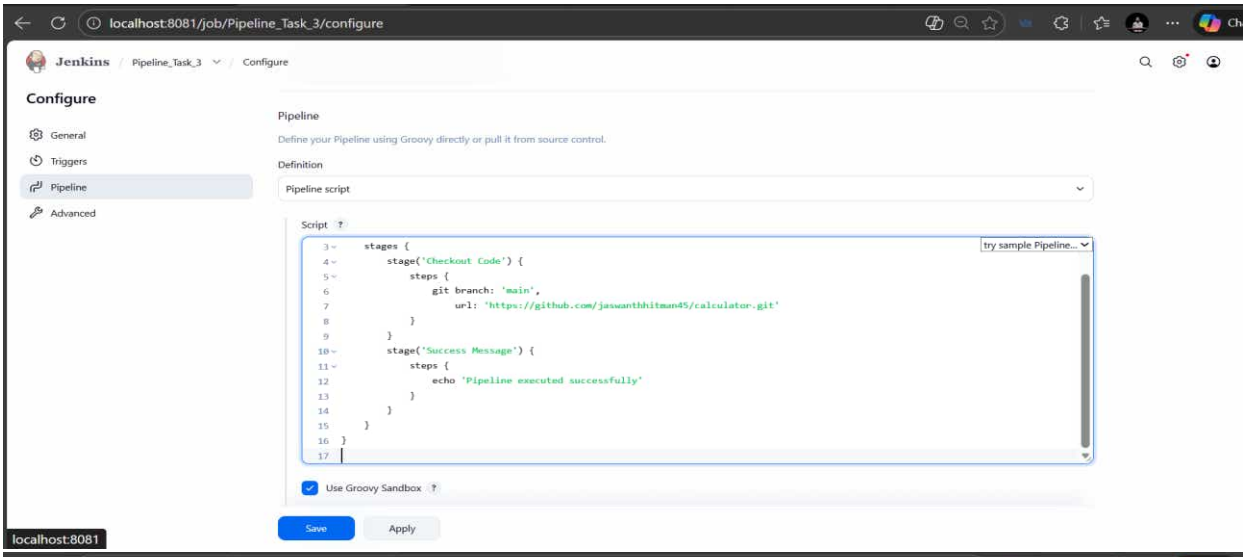
Console Output of Pipeline_Task_2:

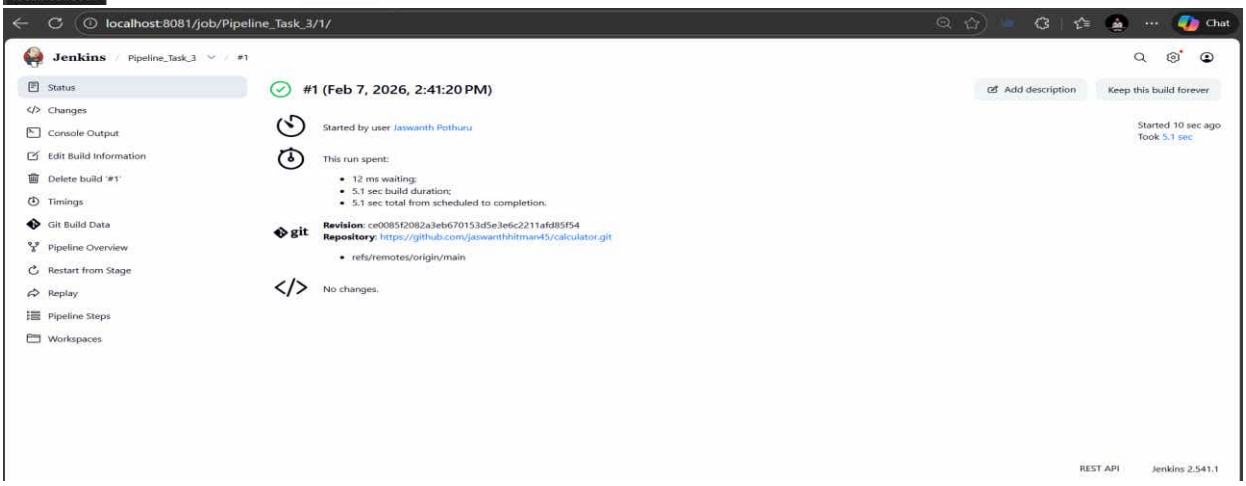


Created Pipeline_Task_3 to checkout code and print a success message using echo.

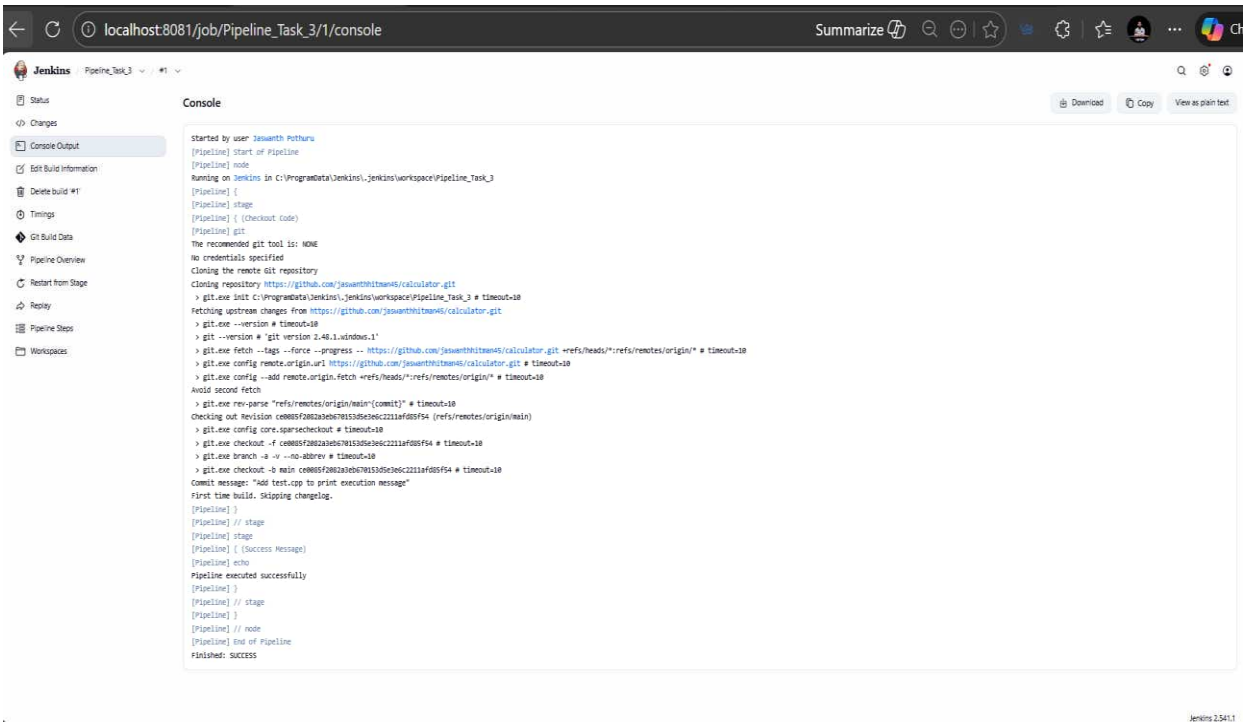


Adding Pipeline Script to Display Success Message:

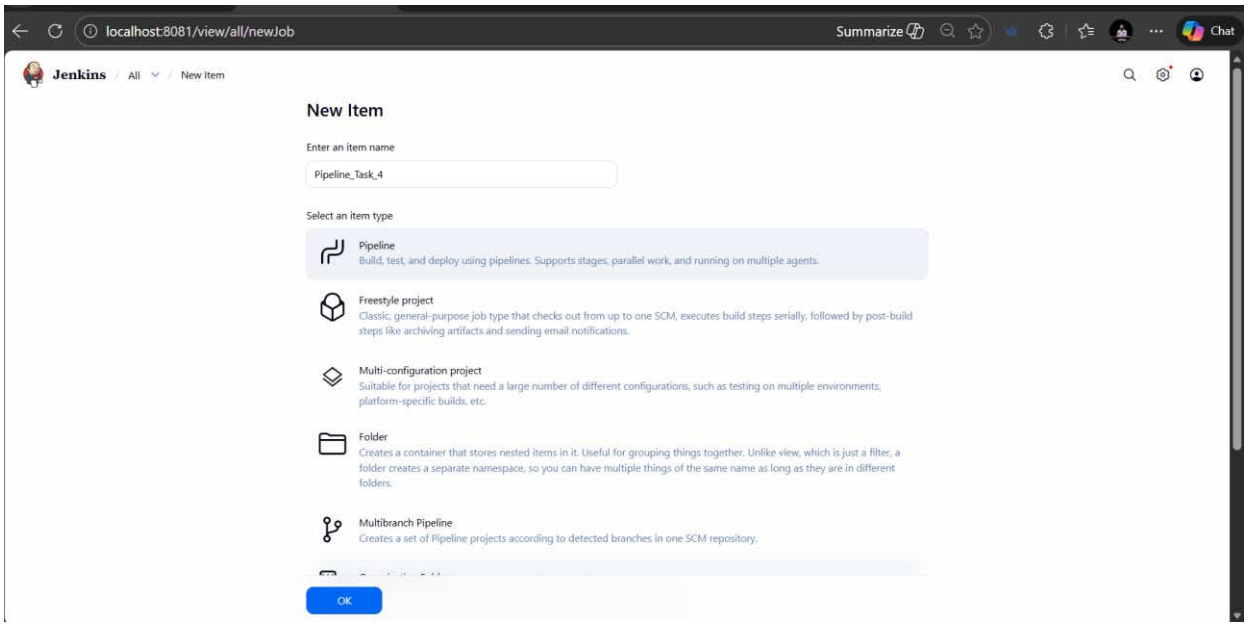




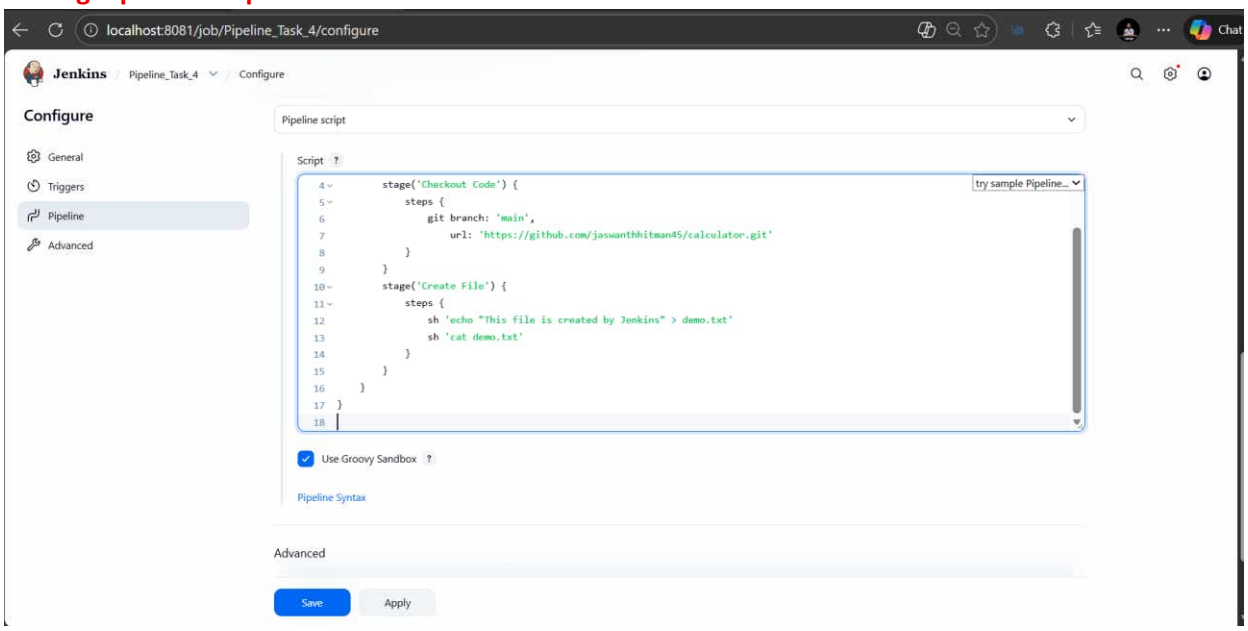
Console Output of Pipeline_Task_3:

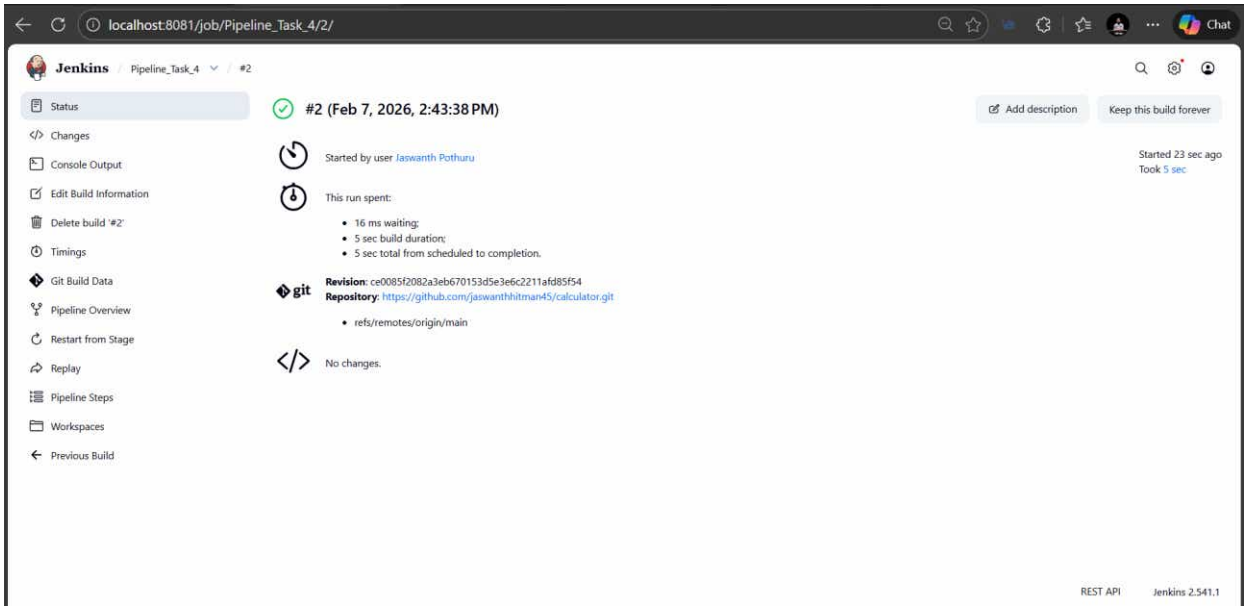


Created Pipeline_Task_4 to checkout code and generate a file demo.txt using Jenkins.

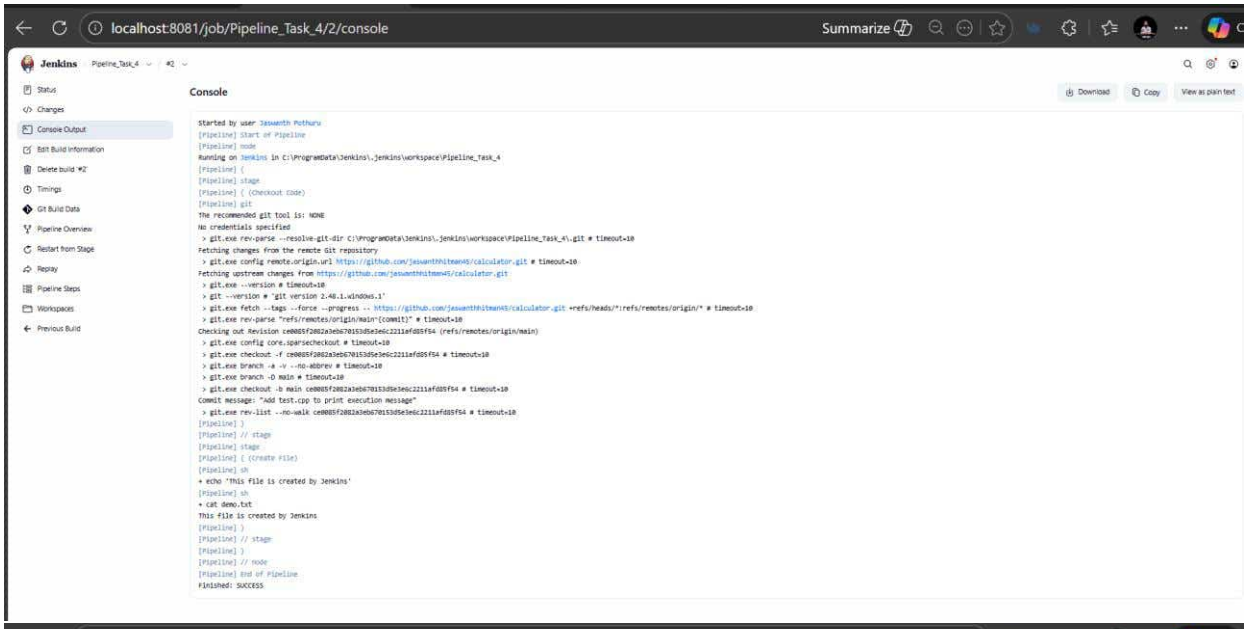


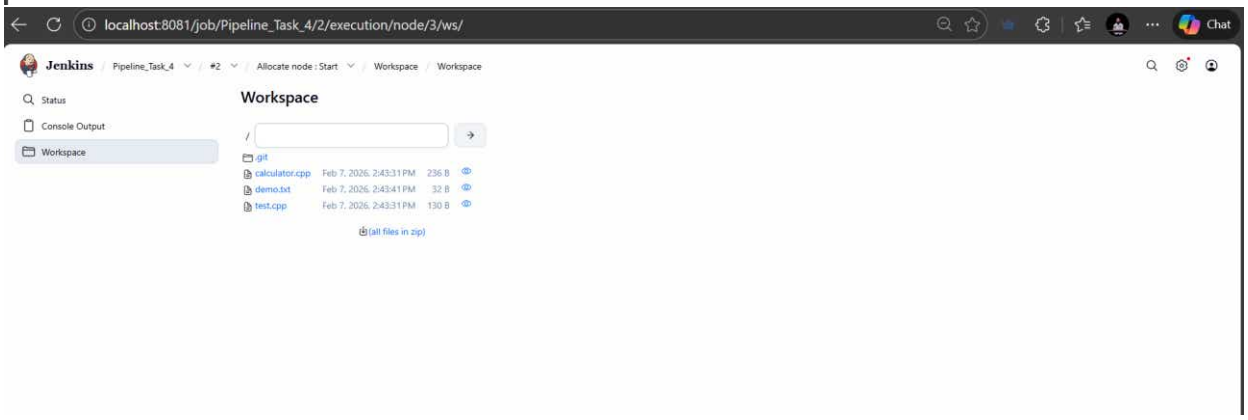
Adding Pipeline Script to Create File:



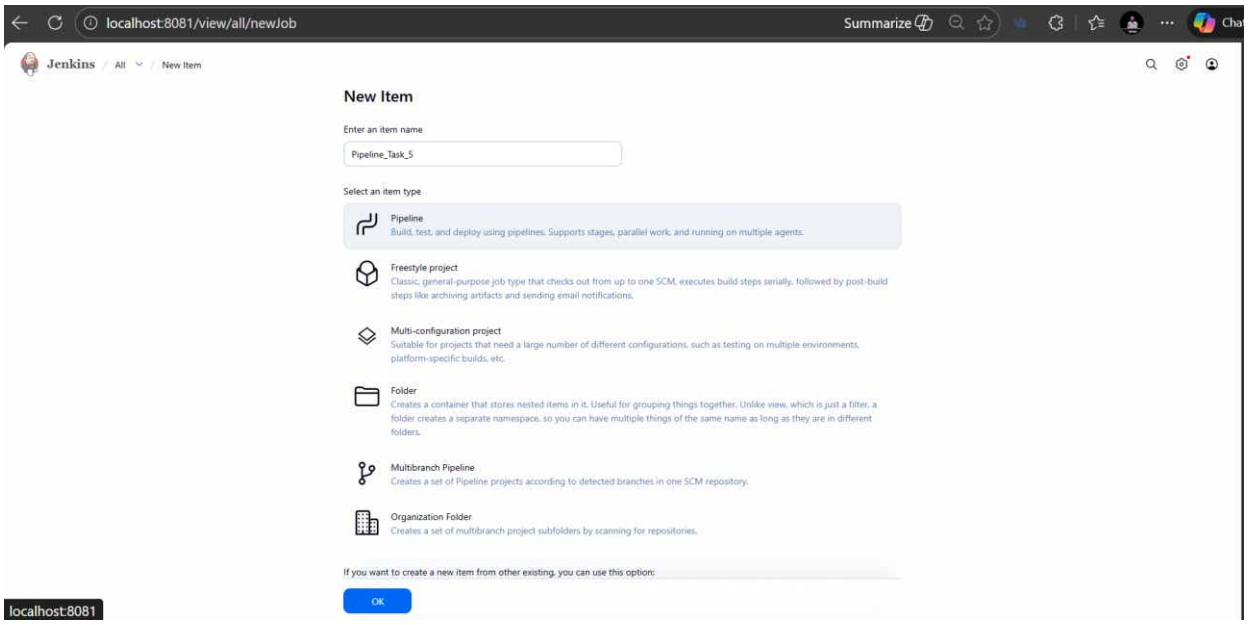


Console Output of Pipeline_Task:

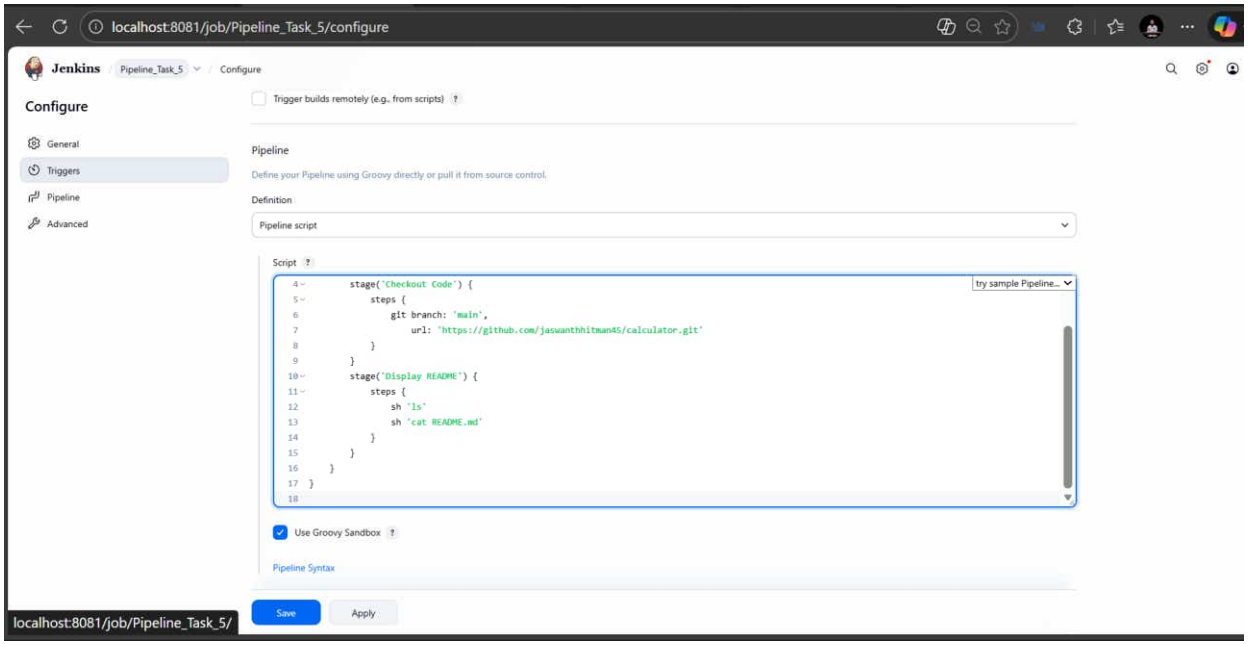


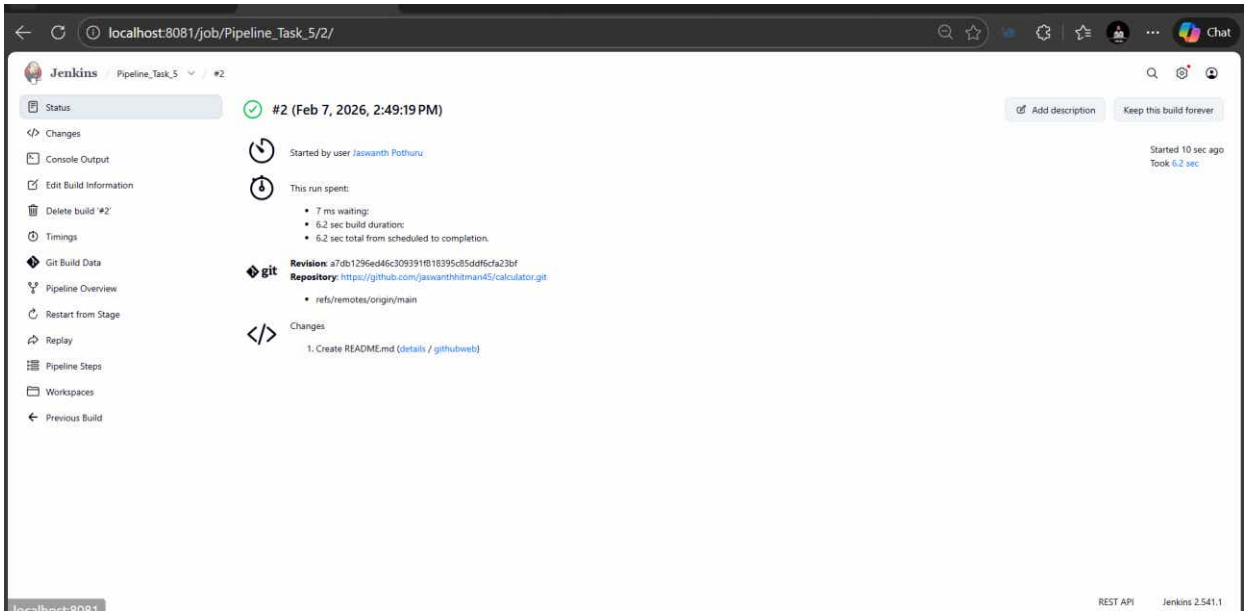
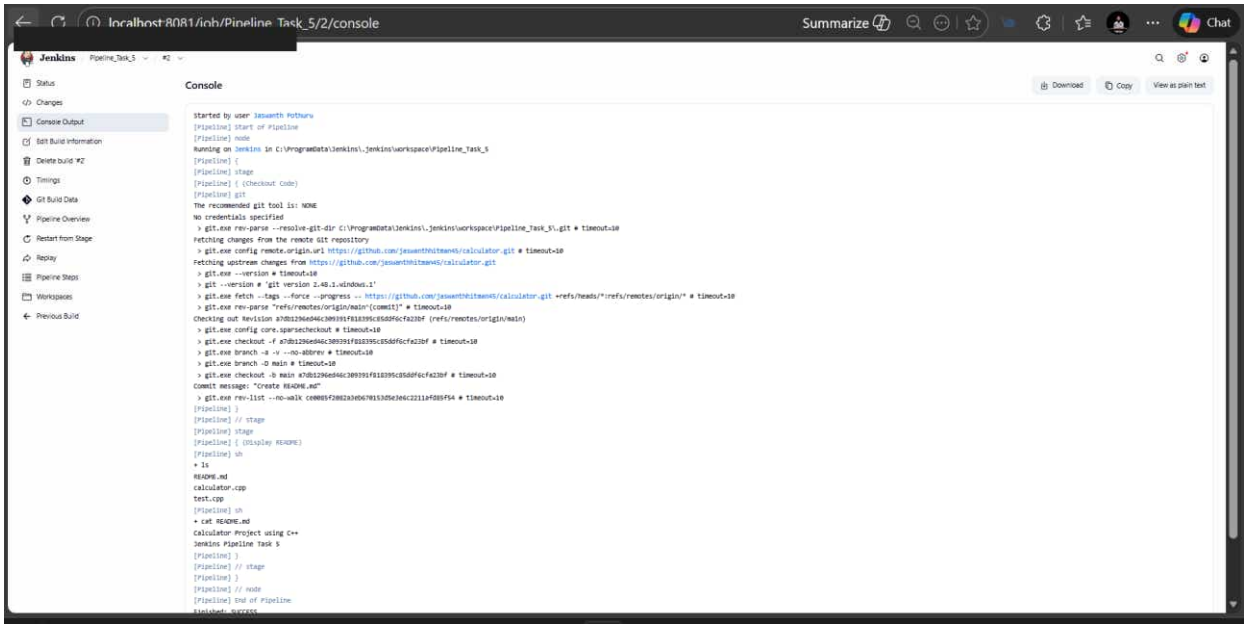


Created Pipeline_Task_5 to checkout code and display the contents of README.md.



Adding Pipeline Script to Display README:





Console Output of Pipeline_Task_5: